Functional Programming in Scala

PAUL CHIUSANO
RÚNAR BJARNASON

MANNING
SHELTER ISLAND

For online information and ordering of this and other Manning books, please visit www.manning.com. The publisher offers discounts on this book when ordered in quantity. For more information, please contact

Special Sales Department
Manning Publications Co.
20 Baldwin Road
PO Box 761
Shelter Island, NY 11964
Email: orders@manning.com

©2015 by Manning Publications Co. All rights reserved.

No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by means electronic, mechanical, photocopying, or otherwise, without prior written permission of the publisher.

Many of the designations used by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in the book, and Manning Publications was aware of a trademark claim, the designations have been printed in initial caps or all caps.

Recognizing the importance of preserving what has been written, it is Manning's policy to have the books we publish printed on acid-free paper, and we exert our best efforts to that end. Recognizing also our responsibility to conserve the resources of our planet, Manning books are printed on paper that is at least 15 percent recycled and processed without the use of elemental chlorine.

Development editor: Jeff Bleiel
Manning Publications Co. Copyeditor: Benjamin Berg
20 Baldwin Road Proofreader: Katie Tennant PO Box
261 Project editor: Janet Vail
Shelter Island, NY 11964 Typesetter: Dottie
Marsico Illustrator: Chuck Larson Cover designer:
Irene Scala

ISBN 9781617290657
Printed in the United States of America
1 2 3 4 5 6 7 8 9 10 – EBM – 19 18 17 16 15 14

brief contents

PART 1 INTRODUCTION TO FUNCTIONAL PROGRAMMING 1

1 · What is functional programming? 3
2 · Getting started with functional programming in Scala 14
3 · Functional data structures 29
4 · Handling errors without exceptions 48
5 · Strictness and laziness 64
6 · Purely functional state 78

PART 2 FUNCTIONAL DESIGN AND COMBINATOR LIBRARIES 93

7 · Purely functional parallelism 95
8 · Property-based testing 124
9 · Parser combinators 146

PART 3 COMMON STRUCTURES IN FUNCTIONAL DESIGN 173

10 · Monoids 175
11 · Monads 187
12 · Applicative and traversable functors 205

PART 4 EFFECTS AND I/O 227

13 · External effects and I/O 229
14 · Local effects and mutable state 254
15 · Stream processing and incremental I/O 268

contents

preface xv
acknowledgments xvi
about this book xvii

PART 1 INTRODUCTION TO FUNCTIONAL PROGRAMMING 1

1 What is functional programming? 3

1.1 The benefits of FP: a simple example 4

A program with side effects 4 · A functional solution: removing the side effects 6

1.2 Exactly what is a (pure) function? 9

1.3 Referential transparency, purity, and the substitution model 10

1.4 Summary 13

2 Getting started with functional programming in Scala 14

2.1 Introducing Scala the language: an example 15

2.2 Running our program 17

2.3 Modules, objects, and namespaces 18

2.4 Higher-order functions: passing functions to functions 19

A short detour: writing loops functionally 20 · Writing our first higher-order function 21

2.5 Polymorphic functions: abstracting over types 22

An example of a polymorphic function 23 · Calling HOFs with anonymous functions 24

2.6 Following types to implementations 25

2.7 Summary 28

3 Functional data structures 29

3.1 Defining functional data structures 29

3.2 Pattern matching 32

3.3 Data sharing in functional data structures 35

The efficiency of data sharing 36 · Improving type inference for higher-order functions 37

3.4 Recursion over lists and generalizing to higher-order functions 38

More functions for working with lists 41 · Loss of efficiency when assembling list functions from simpler components 44

3.5 Trees 44

3.6 Summary 47

4 Handling errors without exceptions 48

4.1 The good and bad aspects of exceptions 48

4.2 Possible alternatives to exceptions 50

4.3 The Option data type 52

Usage patterns for Option 53 ▪ Option composition, lifting, and wrapping exception-oriented APIs 56

4.4 The Either data type 60

4.5 Summary 63

5 Strictness and laziness 64

5.1 Strict and non-strict functions 65

5.2 An extended example: lazy lists 68

Memoizing streams and avoiding recomputation 69 ▪ Helper functions for inspecting streams 69

5.3 Separating program description from evaluation 70

5.4 Infinite streams and corecursion 73

5.5 Summary 77

6 Purely functional state 78

6.1 Generating random numbers using side effects 78

6.2 Purely functional random number generation 80

6.3 Making stateful APIs pure 81

6.4 A better API for state actions 84

Combining state actions 85 ▪ Nesting state actions 86

6.5 A general state action data type 87

6.6 Purely functional imperative programming 88

6.7 Summary 91

PART 2 FUNCTIONAL DESIGN AND COMBINATOR LIBRARIES...93

7 Purely functional parallelism 95

7.1 Choosing data types and functions 96

A data type for parallel computations 97 ▪ Combining parallel computations 100 ▪ Explicit forking 102

7.2 Picking a representation 104

7.3 Refining the API 105

7.4 The algebra of an API 110

The law of mapping 110 ▪ The law of forking 112 Breaking the law: a subtle bug 113 ▪ A fully non-blocking Par implementation using actors 115

7.5 Refining combinators to their most general form 120

7.6 Summary 123

8 Property-based testing 124

8.1 A brief tour of property-based testing 124

8.2 Choosing data types and functions 127

Initial snippets of an API 127 · The meaning and API of properties 128 · The meaning and API of generators 130 Generators that depend on generated values 131 · Refining the Prop data type 132

8.3 Test case minimization 134

8.4 Using the library and improving its usability 136

Some simple examples 137 · Writing a test suite for parallel computations 138

8.5 Testing higher-order functions and future directions 142

8.6 The laws of generators 144

8.7 Summary 144

9 Parser combinators 146

9.1 Designing an algebra, first 147

9.2 A possible algebra 152

Slicing and nonempty repetition 154

9.3 Handling context sensitivity 156

9.4 Writing a JSON parser 158

The JSON format 158 · A JSON parser 159

9.5 Error reporting 160

A possible design 161 · Error nesting 162 Controlling branching and backtracking 163

9.6 Implementing the algebra 165

One possible implementation 166 · Sequencing parsers 166 Labeling parsers 167 · Failover and backtracking 168 Context-sensitive parsing 169

9.7 Summary 171

PART 3COMMON STRUCTURES IN FUNCTIONAL DESIGN.....173

10 Monoids 175

10.1 What is a monoid? 175

10.2 Folding lists with monoids 178

10.3 Associativity and parallelism 179

10.4 Example: Parallel parsing 181

10.5 Foldable data structures 183

10.6 Composing monoids 184

Assembling more complex monoids 185 · Using composed

monoids to fuse traversals 186

10.7 Summary 186

11 Monads 187

11.1 Functors: generalizing the map function 187

Functor laws 189

11.2 Monads: generalizing the flatMap and unit functions 190 *The Monad trait 191*

11.3 Monadic combinators 193

11.4 Monad laws 194

The associative law 194 · Proving the associative law for a specific monad 196 · The identity laws 197

11.5 Just what is a monad? 198

The identity monad 199 · The State monad and partial type application 200

11.6 Summary 204

12 Applicative and traversable functors 205

12.1 Generalizing monads 205

12.2 The Applicative trait 206

12.3 The difference between monads and applicative functors 208

The Option applicative versus the Option monad 209
The Parser applicative versus the Parser monad 210

12.4 The advantages of applicative functors 211

Not all applicative functors are monads 211

12.5 The applicative laws 214

Left and right identity 214 · Associativity 215
Naturality of product 216

12.6 Traversable functors 218

12.7 Uses of Traverse 219

From monoids to applicative functors 220 · Traversals with State 221 · Combining traversable structures 223 · Traversal fusion 224 · Nested traversals 224 · Monad composition 225

12.8 Summary 226

PART 4EFFECTS AND I/O 227

13 External effects and I/O 229

13.1 Factoring effects 229

13.2 A simple IO type 231

Handling input effects 232 ▪ Benefits and drawbacks of the simple IO type 235

13.3 Avoiding the StackOverflowError 237

Reifying control flow as data constructors 237
Trampolining: a general solution to stack overflow 239

13.4 A more nuanced IO type 241

Reasonably priced monads 242 ▪ A monad that supports only console I/O 243 ▪ Pure interpreters 246

13.5 Non-blocking and asynchronous I/O 247

13.6 A general-purpose IO type 250

The main program at the end of the universe 250

13.7 Why the IO type is insufficient for streaming I/O 251 13.8 Summary 253

14 *Local effects and mutable state 254*

14.1 Purely functional mutable state 254

14.2 A data type to enforce scoping of side effects 256

A little language for scoped mutation 256 ▪ An algebra of mutable references 258 ▪ Running mutable state actions 259
Mutable arrays 262 ▪ A purely functional in-place quicksort 263

14.3 Purity is contextual 264

What counts as a side effect? 266

14.4 Summary 267

15 *Stream processing and incremental I/O 268*

15.1 Problems with imperative I/O: an example 268

15.2 Simple stream transducers 271

Creating processes 272 ▪ Composing and appending processes 275 ▪ Processing files 278

15.3 An extensible process type 278

Sources 281 ▪ Ensuring resource safety 283 ▪ Single-input processes 285 ▪ Multiple input streams 287 ▪ Sinks 290
Effectful channels 291 ▪ Dynamic resource allocation 291

15.4 Applications 292

15.5 Summary 293

index 295

foreword

Functional Programming in Scala is an intriguing title. After all, Scala is generally called a functional programming language and there are dozens of books about Scala on the market. Are all these other books missing the functional aspects of the language? To answer the question it's instructive to dig a bit deeper. What is *functional programming*? For me, it's simply an alias for "programming with functions," that is, a programming style that puts the focus on the functions in a program. What are *func tions*? Here, we find a larger spectrum of definitions. While one definition often admits functions

that may have side effects in addition to returning a result, pure functional programming restricts functions to be as they are in mathematics: binary relations that map arguments to results.

Scala is an impure functional programming language in that it admits impure as well as pure functions, and in that it does not try to distinguish between these catego ries by using different syntax or giving them different types. It shares this property with most other functional languages. It would be nice if we could distinguish pure and impure functions in Scala, but I believe we have not yet found a way to do so that is lightweight and flexible enough to be added to Scala without hesitation.

To be sure, Scala programmers are generally encouraged to use pure functions. Side effects such as mutation, I/O, or use of exceptions are not ruled out, and they can indeed come in quite handy sometimes, be it for reasons of interoperability, effi ciency, or convenience. But overusing side effects is generally not considered good style by experts. Nevertheless, since impure programs are possible and even conve nient to write in Scala, there is a temptation for programmers coming from a more imperative background to keep their style and not make the necessary effort to adapt to the functional mindset. In fact, it's quite possible to write Scala as if it were Java without the semicolons.

So to properly learn functional programming in Scala, should one make a detour via a pure functional language such as Haskell? Any argument in favor of this

approach has been severely weakened by the appearance of *Functional Programming in Scala.*

What Paul and Rúnar do, put simply, is treat Scala as a pure functional program ming language. Mutable variables, exceptions, classical input/output, and all other traces of impurity are eliminated. If you wonder how one can write useful programs without any of these conveniences, you need to read the book. Building up from first principles and extending all the way to incremental input and output, they demon strate that, indeed, one can express every concept using only pure functions. And they show that it is not only possible, but that it also leads to beautiful code and deep insights into the nature of computation.

The book is challenging, both because it demands attention to detail and because it might challenge the way you think about programming. By reading the book and doing the recommended exercises, you will develop a better appreciation of what pure functional programming is, what it can express, and what its benefits are.

What I particularly liked about the book is that it is self-contained. It starts with the simplest possible expressions and every abstraction is explained in detail before fur ther abstractions are built on them in turn. In a sense, the book develops an alterna tive Scala universe, where mutable state does not exist and all functions are pure. Commonly used Scala libraries tend to deviate a bit from this ideal; often they are based on a partly imperative implementation with a (mostly) functional interface. That Scala allows the encapsulation of mutable state in a functional interface is, in my opinion, one of its strengths. But it is a capability that is also often misused. If you find yourself using it too often, *Functional Programming in Scala* is a powerful antidote.

MARTIN ODERSKY
CREATOR OF SCALA

preface

Writing good software is hard. After years of struggling with other approaches, both of us discovered and fell in love with functional programming (FP). Though the FP approach is different, we came to appreciate how the discipline leads to a coherent, composable, and beautiful way of writing programs.

Both of us participated in the Boston Area Scala Enthusiasts, a group that met reg ularly in Cambridge. When the group first started, it mainly consisted of Java pro grammers who were looking for something better. Many expressed frustration that there wasn't a clear way to learn how to take advantage of FP in Scala. We could empa thize—we had both learned FP somewhat haphazardly, by writing lots of functional code, talking to and learning from other Scala and Haskell programmers, and reading a patchwork of different articles, blog posts, and books. It felt like there should be an easier way. In April 2010 one of the group's organizers, Nermin Šerifovic´, suggested that we write a book specifically on the topic of FP in Scala. Based on our learning experiences, we had a clear idea of the kind of book we wanted to write, and we thought it would be quick and easy. More than four years later, we think we have cre ated a good book. It's the book we wish had existed when we were learning functional programming.

We hope to convey in this book some of the excitement that we felt when we were first discovering FP.

acknowledgments

We would like to thank the many people who participated in the creation of this book. To Nermin Šerifovic´, our friend from the Boston Scala group, thank you for first planting the seed of this book in our minds.

To the amazing team at Capital IQ, thank you for your support and for bravely helping beta test the first version of the book's curriculum.

We would like to especially acknowledge Tony Morris for embarking on this jour ney with us. His work on the early stages of the book remains invaluable, as does his larger contribution to the practice of functional programming in Scala.

Martin, thank you for your wonderful foreword, and of course for creating this powerful language that is helping to reshape our industry.

During the book-writing process, we were grateful for the encouragement from the enthusiastic community of Scala users

interested in functional programming. To our reviewers, MEAP readers, and everyone else who provided feedback or submitted bug reports and pull requests, thank you! This book would not be what it is today with out your help.

Special thanks to our development editor Jeff Bleiel, our graphics editor Ben Kovitz, our technical proofreader Tom Lockney, and everyone else at Manning who helped make this a better book, including the following reviewers who read the manuscript at various stages of its development: Ashton Hepburn, Bennett Andrews, Chris Marshall, Chris Nauroth, Cody Koeninger, Dave Cleaver, Douglas Alan, Eric Torreborre, Erich W. Schreiner, Fernando Dobladez, Ionut G. Stan, Jeton Bacaj, Kai Gellien, Luc Duponcheel, Mark Janssen, Michael Smolyak, Ravindra Jaju, Rintcius Blok, Rod Hilton, Sebastien Nichele, Sukant Hajra, Thad Meyer, Will Hayworth, and William E. Wheeler.

Lastly, Sunita and Allison, thank you so much for your support throughout our multi-year odyssey to make this book a reality.

about this book

This is not a book about Scala. This book is an introduction to *functional programming* (FP), a radical, principled approach to writing software. We use Scala as the vehicle to get there, but you can apply the lessons herein to programming in any language. As you work through this book, our goal is for you to gain a firm grasp of functional pro gramming concepts, become comfortable writing purely functional programs, and be able to absorb new material on the subject, beyond what we cover here.

How this book is structured

The book is organized into four parts. In part 1, we talk about exactly what functional programming is and introduce some core concepts. The chapters in part 1 give an overview of fundamental techniques like how to organize small functional programs, define purely functional data structures, handle errors, and deal with state.

Building on this foundation, part 2 is a series of tutorials on *functional design*. We work through some examples of practical functional libraries, laying bare the thought process that goes into designing them.

While working through the libraries in part 2, it will become clear to you that these libraries follow certain patterns and contain some duplication. This will highlight the need for new and higher abstractions for writing more generalized libraries, and we introduce those abstractions in part 3. These are very powerful tools for reasoning about your code. Once you master them, they hold the promise of making you extraordinarily productive as a programmer.

Part 4 then bridges the remainder of the gap towards writing real-world applica tions that perform I/O (like working with databases, files, or video displays) and make use of mutable state, all in a purely functional way.

Throughout the book, we rely heavily on programming exercises, carefully sequenced to help you internalize the material. To understand functional program ming, it's not enough to learn the theory abstractly. You have to fire up your text editor

and write some code. You have to take the theory that you have learned and put it into practice in your work.

We've also provided online notes for all the chapters. Each chapter has a section with discussion related to that chapter, along with links to further material. These chapter notes are meant to be expanded by the community of readers, and are avail able as an editable wiki at https://github.com/fpinscala/fpinscala/wiki.

Audience

This book is intended for readers with at least some programming experience. We had a particular kind of reader in mind while writing the book—an intermediate-level Java or C programmer who is curious about functional programming. But we believe this book is well suited for programmers coming from any language, at any level of experience.

Prior expertise is not as important as motivation and curiosity. Functional pro gramming is a lot of fun, but it's a challenging topic. It may be especially challenging for the most experienced programmers, because it requires such a different way of thinking than they might be used to. No matter how long you have been program ming, you must come prepared to be a beginner once again.

This book does not require any prior experience with Scala, but we won't spend a lot of time and space discussing Scala's syntax and language features. Instead we will introduce them as we go, with minimal ceremony, mostly as a consequence of cover ing other material. These introductions to Scala should be enough to get you started with the exercises. If you have further questions about the Scala language, you should supplement your reading with another book on Scala (http://scala-lang.org/ documentation/books.html) or look up specific questions in the Scala language doc umentation (http://scala-lang.org/documentation/).

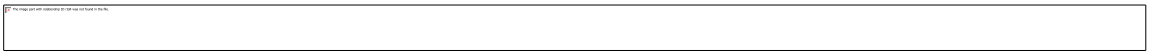
How to read this book

Although the book can be read sequentially straight through, the sequencing of the four parts is designed so that you can comfortably break between them, apply what you have learned to your own work, and then come back later for the next part. For example, the material in part 4 will make the most sense after you have a strong famil iarity with the functional style of programming developed in parts 1, 2, and 3. After part 3, it may be a good idea to take a break and try getting more practice writing functional programs beyond the exercises we work on in the chapters. Of course, this is ultimately up to you.

Most chapters in this book have a similar structure. We introduce some new idea or technique, explain it with an example, and then work through a number of exercises. We *strongly* suggest that you download the exercise source code and do the exercises as you go through each chapter. Exercises, hints, and answers are all available at https:// github.com/fpinscala/fpinscla. We also encourage you to visit the scala-functional

Google group (https://groups.google.com/forum/#!topic/scala-functional/) and the `#fp-in-scala` IRC channel on `irc.freenode.net` for questions and discussion. Exercises are marked for both their difficulty and importance. We will mark exer cises that we think are *hard* or that we

consider to be *optional* to understanding the material. The *hard* designation is our effort to give you some idea of what to expect— it is only our guess and you may find some unmarked questions difficult and some questions marked *hard* to be quite easy. The *optional* designation is for exercises that are informative but can be skipped without impeding your ability to follow further material. The exercises have the following icons in front of them to denote whether or not they are optional:



EXERCISE 1 A filled-in square next to an exercise means the exercise is critical.



EXERCISE 2 An open square means the exercise is optional.

Examples are presented throughout the book, and they are meant to be *tried* rather than just read. Before you begin, you should have the Scala interpreter running and ready. We encourage you to experiment on your own with variations of what you see in the examples. A good way to understand something is to change it slightly and see how the change affects the outcome.

Sometimes we will show a Scala interpreter session to demonstrate the result of running or evaluating some code. This will be marked by lines beginning with the `scala>` prompt of the interpreter. Code that follows this prompt is to be typed or pasted into the interpreter, and the line just below will show the interpreter's response, like this:

```
scala> println("Hello, World!")
Hello, World!
```

Code conventions and downloads

All source code in listings or in text is in a `fixed-width font like this` to separate it from ordinary text. Key words in Scala are set in **`bold fixed-width font like this`**. Code annotations accompany many of the listings, highlighting important concepts.

To download the source code for the examples in the book, the exercise code, and the chapter notes, please go to https://github.com/fpinscala/fpinscala or to the pub lisher's website at www.manning.com/FunctionalProgramminginScala.

Setting expectations

Although functional programming has a profound impact on the way we write soft ware at every level, it takes time to build up to that. It's an incremental process. Don't expect to be blown away by how amazing functional programming is right in the first chapter. The principles laid out in the beginning are quite subtle, and may even seem like they're just common sense. If you think to yourself "that's something I can already do without knowing FP," then that's great! That's exactly the point. Most programmers are already doing FP to some extent, without even knowing it. Many things that most people consider to be best practices (like making a function have only a single respon sibility, or making data immutable) are implied by accepting the premise of functional programming. We are simply taking the principles underlying those best practices and carrying them all the way to their logical conclusion.

It's highly likely that in reading this book, you will simultaneously learn both Scala syntax and functional programming. As a result, at first it may seem to you that the code looks very alien, the techniques are unnatural, and the exercises are brain bending. That is perfectly normal. Do not be deterred. If you get stuck, look at the hints and answers,[1] or take your questions to the Google Group (https://groups .google.com/forum/#!topic/scala-functional/) or the IRC channel (`#fp-in-scala` on `irc.freenode.net`).

Above all, we hope that this book will be a fun and rewarding experience for you, and that functional programming makes your work easier and more enjoyable as it has done for us. This book's purpose, when all is said and done, is to help you be more pro ductive in your work. It should make you feel less like the software you are writing is a collection of dirty hacks, and more like you are creating a thing of beauty and utility.

Author Online

Purchase of *Functional Programming in Scala* includes free access to a private web forum run by Manning Publications where you can make comments about the book, ask technical questions, and receive help from the authors and other users. To access the forum and subscribe to it, point your web browser to www.manning.com/Functional ProgramminginScala. This Author Online page provides information on how to get on the forum once you're registered, what kind of help is available, and the rules of conduct on the forum.

Manning's commitment to our readers is to provide a venue where a meaningful dialog among individual readers and between readers and the authors can take place. It's not a commitment to any specific amount of participation on the part of the authors, whose contribution to the forum remains voluntary (and unpaid).

The Author Online forum and the archives of previous discussions will be accessi ble from the publisher's website as long as the book is in print.

[1] https://github.com/fpinscala/fpinscala.

Part 1

Introduction to functional programming

We begin this book with a radical premise—that we will restrict ourselves to constructing programs using only pure functions with no side effects such as reading from files or mutating memory. This idea, of functional programming, leads to a very different way of writing programs than you may be

used to. We therefore start from the very beginning, relearning how to write the simplest of programs in a functional way.

In the first chapter, we'll explain exactly what functional programming means and give you some idea of its benefits. The rest of the chapters in part 1 introduce the basic techniques for functional programming in Scala. Chapter 2 introduces Scala the language and covers fundamentals like how to write loops functionally and manipulate functions as ordinary values. Chapter 3 deals with in-memory data structures that may change over time. Chapter 4 talks about handling errors in pure functions, and chapter 5 introduces the notion of non strictness, which can be used to improve the efficiency and modularity of func tional code. Finally, chapter 6 introduces modeling stateful programs using pure functions.

The intent of this first part of the book is to get you thinking about programs purely in terms of functions from inputs to outputs, and to teach you the tech niques you'll need in part 2, when we start writing some practical code.

What is functional programming?

Functional programming (FP) is based on a simple premise with far-reaching impli cations: we construct our programs using only *pure functions*—in other words, func tions that have no *side effects*. What are side effects? A function has a side effect if it does something other than simply return a result, for example:

- Modifying a variable
- Modifying a data structure in place
- Setting a field on an object
- Throwing an exception or halting with an error
- Printing to the console or reading user input
- Reading from or writing to a file
- Drawing on the screen

We'll provide a more precise definition of side effects later in this chapter, but con sider what programming would be like without the ability to do these things, or with significant restrictions on when and how these actions can occur. It may be dif ficult to imagine. How is it even possible to write useful programs at all? If we can't reassign variables, how do we write simple programs like loops? What about work ing with data that changes, or handling errors without throwing exceptions? How can we write programs that must perform I/O, like drawing to the screen or read ing from a file?

The answer is that functional programming is a restriction on *how* we write pro grams, but not on *what* programs we can express. Over the course of this book, we'll learn how to express *all* of our programs without side effects, and that includes programs that perform I/O, handle errors, and modify data. We'll learn

how following the discipline of FP is tremendously beneficial because of the increase in *modularity* that we gain from programming with pure functions. Because of their modularity, pure functions are easier to test, reuse, parallelize, generalize, and reason about. Furthermore, pure functions are much less prone to bugs.

In this chapter, we'll look at a simple program with side effects and demonstrate some of the benefits of FP by removing these side effects. We'll also discuss the bene fits of FP more generally and define two important concepts—*referential transparency* and the *substitution model*.

1.1 The benefits of FP: a simple example

Let's look at an example that demonstrates some of the benefits of programming with pure functions. The point here is just to illustrate some basic ideas that we'll return to throughout this book. This will also be your first exposure to Scala's syntax. We'll talk through Scala's syntax much more in the next chapter, so don't worry too much about following every detail. As long as you have a basic idea of what the code is doing, that's what's important.

1.1.1 A program with side effects

Suppose we're implementing a program to handle purchases at a coffee shop. We'll begin with a Scala program that uses side effects in its implementation (also called an *impure* program).

The `class` keyword introduces a class, much like in Java. Its body is contained in curly braces, { and }.

```
class Cafe {

def buyCoffee(cc: CreditCard): Coffee = {
```

Side effect.
A method of a class is introduced by the `def` keyword.
`cc: CreditCard` defines a parameter named `cc` of type `CreditCard`. The `Coffee` return type of the `buyCoffee` method is

Actually charges the credit card.

```
}
val cup = new Coffee() cc.charge(cup.price) cup
}
```

given after the parameter list, and the method body consists of a block within curly braces after an = sign.

No semicolons are necessary. Newlines delimit statements in a block.

We don't need to say `return`. Since

`cup` is the last statement in the block, it is automatically returned.

The line `cc.charge(cup.price)` is an example of a side effect. Charging a credit card involves some interaction with the outside world—suppose it requires contacting the credit card company via some web service, authorizing the transaction, charging the

card, and (if successful) persisting some record of the transaction for later reference. But our function merely returns a `Coffee` and these other actions are happening *on the side*, hence the term "side effect." (Again, we'll define side effects more formally later in this chapter.)

As a result of this side effect, the code is difficult to test. We don't want our tests to actually contact the credit card company and charge the card! This lack of testability is suggesting a design change: arguably, `CreditCard` shouldn't have any knowledge baked into it about how to contact the credit card company to actually execute a charge, nor should it have knowledge of how to persist a record of this charge in our internal systems. We can make the code more modular and testable by letting `Credit Card` be ignorant of these concerns and passing a `Payments` object into `buyCoffee`.

```
class Cafe {
  def buyCoffee(cc: CreditCard, p: Payments): Coffee = {
    val cup = new Coffee()
    p.charge(cc, cup.price)
    cup
  }
}
```

Though side effects still occur when we call `p.charge(cc, cup.price)`, we have at least regained some testability. `Payments` can be an interface, and we can write a mock implementation of this interface that is suitable for testing. But that isn't ideal either. We're forced to make `Payments` an interface, when a concrete class may have been fine otherwise, and any mock implementation will be awkward to use. For example, it might contain some internal state that we'll have to inspect after the call to `buy Coffee`, and our test will have to make sure this state has been appropriately modified (*mutated*) by the call to `charge`. We can use a *mock framework* or similar to handle this detail for us, but this all feels like overkill if we just want to test that `buyCoffee` creates a charge equal to the price of a cup of coffee.

Separate from the concern of testing, there's another problem: it's difficult to reuse `buyCoffee`. Suppose a customer, Alice, would like to order 12 cups of coffee. Ideally we could just reuse `buyCoffee` for this, perhaps calling it 12 times in a loop. But as it is currently implemented, that will involve contacting the payment system 12 times, authorizing 12 separate charges to Alice's credit card! That adds more process ing fees and isn't good for Alice or the coffee shop.

What can we do about this? As the figure at the top of page 6 illustrates, we could write a whole new function, `buyCoffees`, with special logic for batching up the charges.[1] Here, that might not be such a big deal, since the logic of `buyCoffee` is so

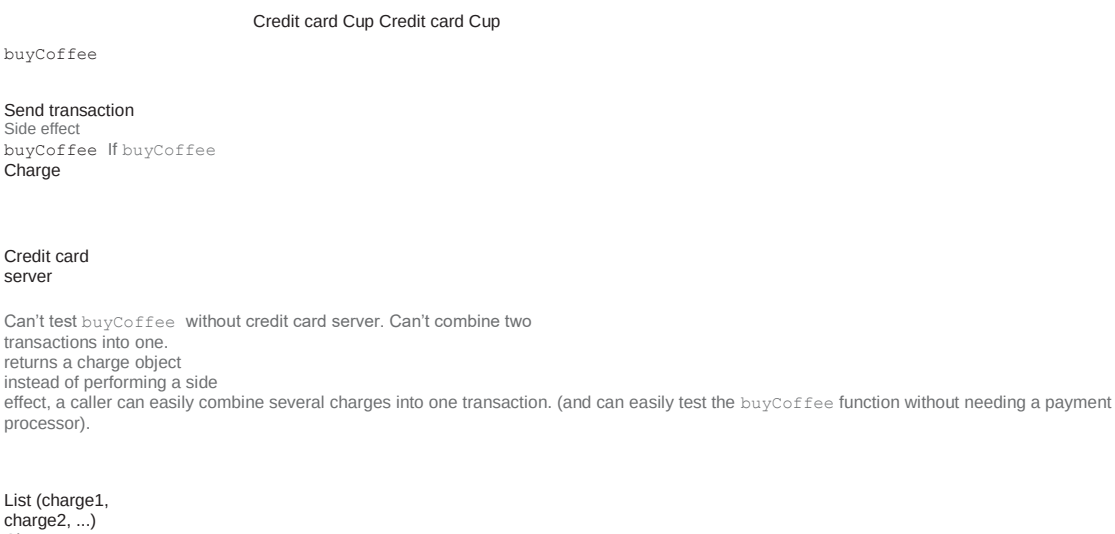
[1] We could also write a specialized `BatchingPayments` implementation of the `Payments` interface, that some how attempts to batch successive charges to the same credit card. This gets complicated though. How many charges should it try to batch up, and how long should

it wait? Do we force `buyCoffee` to indicate that the batch is finished, perhaps by calling `closeBatch`? And how would it know when it's appropriate to do that, anyway?

call to buyCoffee

With a side effect

Without a side effect



simple, but in other cases the logic we need to duplicate may be nontrivial, and we should mourn the loss of code reuse and composition!

1.1.2 A functional solution: removing the side effects

The functional solution is to eliminate side effects and have `buyCoffee` *return the charge as a value* in addition to returning the `Coffee`. The concerns of processing the charge by sending it off to the credit card company, persisting a record of it, and so on, will be handled elsewhere. Again, we'll cover Scala's syntax more in later chapters, but here's what a functional solution might look like:

`buyCoffee` now returns a pair of a

```
class Cafe {
def buyCoffee(cc: CreditCard): (Coffee, Charge) = { val cup = new Coffee()
(cup, Charge(cc, cup.price))
}
```

`Coffee` and a `Charge`, indicated with the type `(Coffee, Charge)`. Whatever system processes payments is not involved at all here. To create a pair, we put the `cup` and `Charge`

```
}
```

in parentheses separated by a comma.

Here we've separated the concern of *creating* a charge from the *processing* or *interpreta tion* of that charge. The `buyCoffee` function now returns a `Charge` as a value along with the `Coffee`. We'll see shortly how this lets us reuse it more easily to purchase mul tiple coffees with a single transaction. But what is `Charge`? It's a data type we just invented containing a `CreditCard` and an `amount`, equipped with a handy function, `combine`, for combining charges with the same `CreditCard`:

A case class has one primary constructor whose argument list comes after the class name (here, `Charge`). The parameters in this list become public, unmodifiable (immutable) fields of the `class` and can be accessed using the usual object-oriented dot notation, as in `other.cc`.

```
case class Charge(cc: CreditCard, amount: Double) {

def combine(other: Charge): Charge = if (cc == other.cc)
      Charge(cc, amount + other.amount) else
```

An `if` expression has the same syntax as in Java, but it also returns a value equal to the result of whichever branch is taken. If `cc == other.cc`, then `combine` will return `Charge(..)`; otherwise the exception in the `else` branch will be thrown.

```
        throw new Exception("Can't combine charges to different cards")

                }
```

The syntax for throwing exceptions is the same as in Java and many other languages. We'll discuss more functional ways of handling error conditions in a later chapter.

A `case class` can be created without the keyword `new`. We just use the class name followed by the list of arguments for its primary constructor.

Now let's look at `buyCoffees`, to implement the purchase of `n` cups of coffee. Unlike before, this can now be implemented in terms of `buyCoffee`, as we had hoped.

`List.fill(n)(x)` creates a `List` with `n` copies of `x`. We'll explain this funny function call syntax in a later chapter.

```
class Cafe {

def buyCoffee(cc: CreditCard): (Coffee, Charge) = ...
```

`List[Coffee]` is an immutable singly linked list of `Coffee` values. We'll discuss this data type more in chapter 3.

```
def buyCoffees(cc: CreditCard, n: Int): (List[Coffee], Charge) = { val purchases:
List[(Coffee, Charge)] = List.fill(n)(buyCoffee(cc)) val (coffees, charges) =
purchases.unzip
(coffees, charges.reduce((c1,c2) => c1.combine(c2)))
 }
}
```

`unzip` splits a list of pairs into a pair of lists. Here we're destructuring this pair to declare two values (`coffees` and `charges`) on one line.

`charges.reduce` reduces the entire list of charges to a single charge, using `combine` to combine charges two at a time. `reduce` is an example of a higher-order function, which we'll properly introduce in the next chapter.

Overall, this solution is a marked improvement—we're now able to reuse `buyCoffee` directly to define the `buyCoffees` function, and both functions are trivially testable without having to define complicated mock implementations of some `Payments` inter face! In fact, the `Cafe` is now completely ignorant of how the `Charge` values will be

processed. We can still have a `Payments` class for actually processing charges, of course, but `Cafe` doesn't need to know about it.

Making `Charge` into a first-class value has other benefits we might not have antici pated: we can more easily assemble business logic for working with these charges. For instance, Alice may bring her laptop to the coffee shop and work there for a few hours, making occasional purchases. It might be nice if the coffee shop could com bine these purchases Alice makes into a single charge, again saving on credit card pro cessing fees. Since `Charge` is first-class, we can write the following function to coalesce any same-card charges in a `List[Charge]`:

```
def coalesce(charges: List[Charge]): List[Charge] =
charges.groupBy(_.cc).values.map(_.reduce(_ combine _)).toList
```

We're passing functions as values to the `groupBy`, `map`, and `reduce` methods. You'll learn to read and write one-liners like this over the next several chapters. The `_.cc` and `_ combine _` are syntax for *anonymous functions*, which we'll introduce in the next chapter.

You may find this kind of code difficult to read because the notation is very com pact. But as you work through this book, reading and writing Scala code like this will become second nature to you very quickly. This function takes a list of charges, groups them by the credit card used, and then combines them into a single charge per card. It's perfectly reusable and testable without any additional mock objects or interfaces. Imagine trying to implement the same logic with our first implementation of `buyCoffee`!

This is just a taste of why functional programming has the benefits claimed, and this example is intentionally simple. If the series of refactorings used here seems natu ral, obvious, unremarkable, or standard practice, that's *good*. FP is merely a discipline that takes what many consider a good idea to its logical endpoint, applying the disci pline even in situations where its applicability is less obvious. As you'll learn over the course of this book, the consequences of consistently following the discipline of FP are profound and the benefits enormous. FP is a truly radical shift in how programs are organized at every level—from the simplest of loops to high-level program architec ture. The style that emerges is quite different, but it's a beautiful and cohesive approach to programming that we hope you come to appreciate.

What about the real world?

We saw in the case of `buyCoffee` how we could separate the creation of the `Charge` from the interpretation or processing of that `Charge`. In general, we'll learn how this sort of transformation can be applied to *any* function with side effects to push these effects to the outer layers of the program. Functional programmers often speak of implementing programs with a pure core and a thin layer on the outside that handles effects.

> But even so, surely at some point we must actually have an effect on the world and submit the `Charge` for processing by some external system. And aren't there other useful programs that necessitate side effects or mutation? How do we write such pro grams? As we work through this book, we'll discover how many programs that seem to necessitate side effects have some functional analogue. In other cases we'll find ways to structure code so that effects occur but aren't *observable*. (For example, we can mutate data that's declared locally in the body of some function if we ensure that it can't be referenced outside that function, or we can write to a file as long as no enclosing function can observe this occurring.)

1.2 Exactly what is a (pure) function?

We said earlier that FP means programming with pure functions, and a pure function is one that lacks side effects. In our discussion of the coffee shop example, we worked off an informal notion of side effects and purity. Here we'll formalize this notion, to pinpoint more precisely what it means to program functionally. This will also give us additional insight into one of the benefits of functional programming: pure functions are easier to reason about.

A function `f` with input type `A` and output type `B` (written in Scala as a single type: `A => B`, pronounced "A to B" or "A arrow B") is a computation that relates every value `a` of type `A` to exactly one value `b` of type `B` such that `b` is determined solely by the value of `a`. Any changing state of an internal or external process is irrelevant to computing the result `f(a)`. For example, a function `intToString` having type `Int => String` will take every integer to a corresponding string. Furthermore, if it really is a *function*, it will do nothing else.

In other words, a function has no observable effect on the execution of the pro gram other than to compute a result given its inputs; we say that it has no side effects. We sometimes qualify such functions as *pure* functions to make this more explicit, but this is somewhat redundant. Unless we state otherwise, we'll often use *function* to imply no side effects.[2]

You should be familiar with a lot of pure functions already. Consider the addition (`+`) function on integers. It takes two integer values and returns an integer value. For any two given integer values, it will *always return the same integer value*. Another example is the `length` function of a `String` in Java, Scala, and many other languages where strings can't be modified (are immutable). For any given string, the same length is always returned and nothing else occurs.

We can formalize this idea of pure functions using the concept of *referential trans parency (RT)*. This is a property of *expressions* in general and not just functions. For the purposes of our discussion, consider an expression to be any part of a program that can be evaluated to a result—anything that you could type into the Scala interpreter

[2] *Procedure* is often used to refer to some parameterized chunk of code that may have side effects.

and get an answer. For example, `2 + 3` is an expression that applies the pure function `+` to the values `2` and `3` (which are also expressions). This has no side effect. The evalu ation of this expression results in the same value `5` every time. In fact, if we saw `2 + 3` in a program we could simply replace it with the value `5` and it wouldn't change a thing about the meaning of our program.

This is all it means for an expression to be referentially transparent—in any pro gram, the expression can be replaced by its result without changing the meaning of the program. And we say that a function is *pure* if calling it with RT arguments is also RT. We'll look at some examples next.

> **Referential transparency and purity**
>
> An expression `e` is *referentially transparent* if, for all programs `p`, all occurrences of `e` in `p` can be replaced by the result of evaluating `e` without affecting the meaning of `p`. A function `f` is *pure* if the expression `f(x)` is referentially transparent for all referen tially transparent `x`.[3]

1.3 Referential transparency, purity, and the substitution model

Let's see how the definition of RT applies to our original `buyCoffee` example:

```
def buyCoffee(cc: CreditCard): Coffee = {
  val cup = new Coffee()
  cc.charge(cup.price)
  cup
}
```

Whatever the return type of `cc.charge(cup.price)` (perhaps it's `Unit`, Scala's equiva lent of `void` in other languages), it's discarded by `buyCoffee`. Thus, the result of eval uating `buyCoffee(aliceCreditCard)` will be merely `cup`, which is equivalent to a `new Coffee()`. For `buyCoffee` to be pure, by our definition of RT, it must be the case that `p(buyCoffee(aliceCreditCard))` behaves the same as `p(new Coffee())`, for *any* `p`. This clearly doesn't hold—the program `new Coffee()` doesn't do anything, whereas `buyCoffee(aliceCreditCard)` will contact the credit card company and authorize a charge. Already we have an observable difference between the two programs.

Referential transparency forces the invariant that everything a function *does* is rep resented by the *value* that it returns, according to the result type of the function. This constraint enables a simple and natural mode of reasoning about program evaluation called the *substitution model*. When expressions are referentially transparent, we can imagine that computation proceeds much like we'd solve an algebraic equation. We fully expand every part of an expression, replacing all variables with their referents, and then reduce it to its simplest form. At each step we replace a term with an

[3] There are some subtleties to this definition, and we'll refine it later in this book. See the chapter notes at our GitHub site (https://github.com/pchiusano/fpinscala; see the preface) for more discussion.

equivalent one; computation proceeds by substituting *equals for equals*. In other words, RT enables *equational reasoning* about programs.

Let's look at two more examples—one where all expressions are RT and can be rea soned about using the substitution model, and one where some expressions violate RT. There's nothing complicated here; we're just formalizing something you likely already understand.

Let's try the following in the Scala interpreter (also known as the Read-Eval-Print Loop or REPL, pronounced like "ripple," but with an *e* instead of an *i*). Note that in Java and in Scala, strings are immutable. A "modified" string is really a new string and the old string remains intact:

```
scala> val x = "Hello, World"
x: java.lang.String = Hello, World

scala> val r1 = x.reverse
r1: String = dlroW ,olleH

scala> val r2 = x.reverse r2: String = dlroW ,olleH
```

`r1` and `r2` are the same.

Suppose we replace all occurrences of the term `x` with the expression referenced by `x` (its *definition*), as follows:

```
scala> val r1 = "Hello, World".reverse
r1: String = dlroW ,olleH

scala> val r2 = "Hello, World".reverse r2: String = dlroW ,olleH
```

`r1` and `r2` are still the same.

This transformation doesn't affect the outcome. The values of `r1` and `r2` are the same as before, so `x` was referentially transparent. What's more, `r1` and `r2` are referentially transparent as well, so if they appeared in some other part of a larger program, they could in turn be replaced with their values throughout and it would have no effect on the program.

Now let's look at a function that is *not* referentially transparent. Consider the `append` function on the `java.lang.StringBuilder` class. This function operates on the `StringBuilder` in place. The previous state of the `StringBuilder` is destroyed after a call to `append`. Let's try this out:

```
scala> val x = new StringBuilder("Hello")
x: java.lang.StringBuilder = Hello

scala> val y = x.append(", World")
y: java.lang.StringBuilder = Hello, World

scala> val r1 = y.toString
r1: java.lang.String = Hello, World

scala> val r2 = y.toString
r2: java.lang.String = Hello, World
```

`r1` and `r2` are the same.

So far so good. Now let's see how this side effect breaks RT. Suppose we substitute the call to `append` like we did earlier, replacing all occurrences of `y` with the expression referenced by `y`:

```
scala> val x = new StringBuilder("Hello")
x: java.lang.StringBuilder = Hello

scala> val r1 = x.append(", World").toString
```

```
                    r1: java.lang.String = Hello, World

scala> val r2 = x.append(", World").toString r2: java.lang.String = Hello, World, World
```

`r1` and `r2` are no longer the same.

This transformation of the program results in a different outcome. We therefore con clude that `StringBuilder.append` is *not* a pure function. What's going on here is that although `r1` and `r2` look like they're the same expression, they are in fact referencing two different values of the same `StringBuilder`. By the time `r2` calls `x.append`, `r1` will have already mutated the object referenced by `x`. If this seems difficult to think about, that's because it is. Side effects make reasoning about program behavior more difficult.

Conversely, the substitution model is simple to reason about since effects of evalua tion are purely local (they affect only the expression being evaluated) and we need not mentally simulate sequences of state updates to understand a block of code. Understanding requires only *local reasoning*. We need not mentally track all the state changes that may occur before or after our function's execution to understand what our function will do; we simply look at the function's definition and substitute the arguments into its body. Even if you haven't used the name "substitution model," you have certainly used this mode of reasoning when thinking about your code.[4]

Formalizing the notion of purity this way gives insight into why functional pro grams are often more modular. Modular programs consist of components that can be understood and reused independently of the whole, such that the meaning of the whole depends only on the meaning of the components and the rules governing their composition; that is, they are *composable*. A pure function is modular and composable because it separates the logic of the computation itself from "what to do with the result" and "how to obtain the input"; it's a black box. Input is obtained in exactly one way: via the argument(s) to the function. And the output is simply computed and returned. By keeping each of these concerns separate, the logic of the computation is more reusable; we may reuse the logic wherever we want without worrying about whether the side effect being done with the result or the side effect requesting the input are appropriate in all contexts. We saw this in the `buyCoffee` example—by elim inating the side effect of payment processing being done with the output, we were more easily able to reuse the logic of the function, both for purposes of testing and for purposes of further composition (like when we wrote `buyCoffees` and `coalesce`).

[4] In practice, programmers don't spend time mechanically applying substitution to determine if code is pure— it will usually be obvious.

1.4 Summary

In this chapter, we introduced functional programming and explained exactly what FP is and why you might use it. Though the full benefits of the functional style will become more clear over the course of this book, we illustrated some of the benefits of FP using a simple example. We also discussed referential transparency and the substi tution model and talked about how FP enables simpler reasoning about programs and greater modularity.

In this book, you'll learn the concepts and principles of FP as they apply to every level of programming, starting from the simplest of tasks and building on that founda tion. In subsequent chapters, we'll cover some of the fundamentals—how do we write loops in FP? Or

implement data structures? How do we deal with errors and excep tions? We need to learn how to do these things and get comfortable with the low-level idioms of FP. We'll build on this understanding when we explore functional design techniques in parts 2, 3, and 4.

Getting started with functional programming in Scala

Now that we have committed to using only pure functions, a question naturally emerges: how do we write even the simplest of programs? Most of us are used to thinking of programs as sequences of instructions that are executed in order, where each instruction has some kind of effect. In this chapter, we'll begin learning how to write programs in the Scala language just by combining pure functions.

This chapter is mainly intended for those readers who are new to Scala, to func tional programming, or both. Immersion is an effective method for learning a for eign language, so we'll just dive in. The only way for Scala code to look familiar and not foreign is if we look at a lot of Scala code. We've already seen some in the first chapter. In this chapter, we'll start by looking at a small but complete program. We'll then break it down piece by piece to examine what it does in some detail, in order to understand the basics of the Scala language and its syntax. Our goal in this book is to teach functional programming, but we'll use Scala as our vehicle, and need to know enough of the Scala language and its syntax to get going.

Once we've covered some of the basic elements of the Scala language, we'll then introduce some of the basic techniques for how to write functional programs. We'll discuss how to write loops using *tail recursive functions*, and we'll introduce *higher order functions (HOFs)*. HOFs are functions that take other functions as arguments and may themselves return functions as their output. We'll also look at some examples of *polymorphic* HOFs where we use types to guide us toward an implementation.

There's a lot of new material in this chapter. Some of the material related to HOFs may be brain-bending if you have a lot of experience programming in a lan guage *without* the ability to pass functions around like that. Remember, it's not cru cial that you internalize every single concept in this chapter, or solve every exercise.

We'll come back to these concepts again from different angles throughout the book, and our goal here is just to give you some initial exposure.

2.1 Introducing Scala the language: an example

The following is a complete program listing in Scala, which we'll talk through. We aren't introducing any new concepts of functional programming here. Our goal is just to introduce the Scala language and its syntax.

Returns the negation of `n` if it's less than zero.

```
// A comment!
/* Another comment */
/** A documentation comment */ object MyModule {
def abs(n: Int): Int =
 if (n < 0) -n
else n


private def formatAbs(x: Int) = {
```

Declares a singleton object, which simultaneously declares a class and its only instance.

`abs` takes an integer and returns an integer.

A private method can only be called by other members of `MyModule`.

```
val msg = "The absolute value of %d is %d" msg.format(x, abs(x))
}


def main(args: Array[String]): Unit = println(formatAbs(-42))
}
```

`Unit` serves the same purpose as `void` in languages like Java or C.

We declare an object (also known as a *module*) named `MyModule`. This is simply to give our code a place to live and a name so we can refer to it later. Scala code has to be in an `object` or a `class`, and we use an `object` here because it's simpler. We put our code inside the object, between curly braces. We'll discuss objects and classes in more detail shortly. For now, we'll just look at this particular object.

The `MyModule` object has three *methods*, intro duced with the `def` keyword: `abs`, `formatAbs`, and `main`. We'll use the term *method* to refer to some function or field defined within an object or class using the `def` keyword. Let's now go through the methods of `MyModule` one by one.

A string with two placeholders for numbers marked as `%d`.

Replaces the two `%d` placeholders in the string with `x` and `abs(x)` respectively.

The object keyword

The `object` keyword creates a new *singleton type*, which is like a `class` that only has a single named instance. If you're familiar with Java, declaring an `object` in Scala is a lot like creating a new instance of an anonymous class.

Scala has no equivalent to Java's `static` keyword, and an `object` is often used in Scala where you might use a class with static members in Java.

The `abs` method is a pure function that takes an integer and returns its absolute value:

```
def abs(n: Int): Int =
  if (n < 0) -n
  else n
```

The `def` keyword is followed by the name of the method, which is followed by the parameter list in parentheses. In this case, `abs` takes only one argument, `n` of type `Int`. Following the closing parenthesis of the argument list, an optional type annotation (the `: Int`) indicates that the type of the result is `Int` (the colon is pronounced "has type").

The body of the method itself comes after a single equals sign (=). We'll sometimes refer to the part of a declaration that goes before the equals sign as the *left-hand side* or *signature*, and the code that comes after the equals sign as the *right-hand side* or *defini tion*. Note the absence of an explicit `return` keyword. The value returned from a method is simply whatever value results from evaluating the right-hand side. All expressions, including `if` expressions, produce a result. Here, the right-hand side is a single expression whose value is either `-n` or `n`, depending on whether `n < 0`.

The `formatAbs` method is another pure function:

```
private def formatAbs(x: Int) = {
val msg = "The absolute value of %d is %d." msg.format(x, abs(x))
}
```

`format` is a standard library method defined on `String`

Here we're calling the `format` method on the `msg` object, passing in the value of `x` along with the value of `abs` applied to `x`. This results in a new string with the occur rences of `%d` in `msg` replaced with the evaluated results of `x` and `abs(x)`, respectively.

This method is declared `private`, which means that it can't be called from any code outside of the `MyModule` object. This function takes an `Int` and returns a `String`, but note that the return type is not declared. Scala is usually able to infer the return types of methods, so they can be omitted, but it's generally considered good style to explicitly declare the return types of methods that you expect

others to use. This method is private to our module, so we can omit the type annotation.

The body of the method contains more than one statement, so we put them inside curly braces. A pair of braces containing statements is called a *block*. Statements are separated by newlines or by semicolons. In this case, we're using a newline to separate our statements, so a semicolon isn't necessary.

The first statement in the block declares a `String` named `msg` using the `val` key word. It's simply there to give a name to the string value so we can refer to it again. A `val` is an immutable variable, so inside the body of the `formatAbs` method the name `msg` will always refer to the same `String` value. The Scala compiler will complain if you try to reassign `msg` to a different value in the same block.

Remember, a method simply returns the value of its right-hand side, so we don't need a `return` keyword. In this case, the right-hand side is a block. In Scala, the value

of a multistatement block inside curly braces is the same as the value returned by the last expression in the block. Therefore, the result of the `formatAbs` method is just the value returned by the call to `msg.format(x, abs(x))`.

Finally, our `main` method is an outer shell that calls into our purely functional core and prints the answer to the console. We'll sometimes call such methods *procedures* (or *impure functions*) rather than functions, to emphasize the fact that they have side effects:

```
def main(args: Array[String]): Unit =
  println(formatAbs(-42))
```

The name `main` is special because when you run a program, Scala will look for a method named `main` with a specific signature. It has to take an `Array` of `String`s as its argument, and its return type must be `Unit`.

The `args` array will contain the arguments that were given at the command line that ran the program. We're not using them here.

`Unit` serves a similar purpose to `void` in programming languages like C and Java. In Scala, every method has to return some value as long as it doesn't crash or hang. But `main` doesn't return anything meaningful, so there's a special type `Unit` that is the return type of such methods. There's only one value of this type and the literal syntax for it is `()`, a pair of empty parentheses, pronounced "unit" just like the type. Usually a return type of `Unit` is a hint that the method has a side effect.

The body of our `main` method prints to the console the `String` returned by the call to `formatAbs`. Note that the return type of `println` is `Unit`, which happens to be what we need to return from `main`.

2.2 Running our program

This section discusses the simplest possible way of running your Scala programs, suit able for short examples. More typically, you'll build and run your Scala code using `sbt`, the build tool for Scala, and/or an IDE like IntelliJ or Eclipse. See the book's source code repo on GitHub (https://github.com/fpinscala/fpinscala) for more information on getting set up with `sbt`.

The simplest way we can run this Scala program (`MyModule`) is from the command line, by invoking the Scala compiler directly ourselves. We start by putting the code in a file called

MyModule.scala or something similar. We can then compile it to Java bytecode using the scalac compiler:

```
> scalac MyModule.scala
```

This will generate some files ending with the .class suffix. These files contain com piled code that can be run with the Java Virtual Machine (JVM). The code can be exe cuted using the scala command-line tool:

```
> scala MyModule
The absolute value of -42 is 42.
```


Actually, it's not strictly necessary to compile the code first with scalac. A simple pro gram like the one we've written here can be run using just the Scala interpreter by passing it to the scala command-line tool directly:

```
> scala MyModule.scala
The absolute value of -42 is 42.
```

This can be handy when using Scala for scripting. The interpreter will look for any object within the file MyModule.scala that has a main method with the appropriate signature, and will then call it.

Lastly, an alternative way is to start the Scala interpreter's interactive mode, the REPL (which stands for read-evaluate-print loop). It's a great idea to have a REPL win dow open so you can try things out while you're programming in Scala.

We can load our source file into the REPL and try things out (your actual console output may differ slightly):

:load is a command to the REPL to interpret a Scala source file. (Note that unfortunately this won't work for Scala files with package declarations.) We can

```
> scala
Welcome to Scala.
Type in expressions to have them evaluated. Type :help for more information.
```

type Scala expressions at the prompt.

```
scala> :load MyModule.scala Loading MyModule.scala... defined module MyModule

scala> MyModule.abs(-42) res0: Int = 42
```

The REPL evaluates our Scala expression and prints the answer. It also gives the answer a name, res0, that we can refer to later, and shows its type, which in this case is Int.

It's also possible to copy and paste individual lines of code into the REPL. It even has a paste mode (accessed with the :paste command) designed to paste code that spans multiple lines. It's a good idea to get familiar with the REPL and its features because it's a tool that you'll use a lot as a Scala programmer.

2.3 Modules, objects, and namespaces

In this section, we'll discuss some additional aspects of Scala's syntax related to mod ules, objects, and namespaces. In the preceding REPL session, note that in order to refer to our abs method, we had to say MyModule.abs because abs was defined in the MyModule object. We say that MyModule is its *namespace*. Aside from some technicalities, every value in Scala is what's called an *object*,[1] and each object may have zero or more *members*. An object whose primary purpose is giving its members a namespace is some times called a *module*. A member can be a method declared with the

`def` keyword, or it can be another object declared with `val` or `object`. Objects can also have other kinds of members that we'll ignore for now.

We access the members of objects with the typical object-oriented dot notation, which is a namespace (the name that refers to the object) followed by a dot (the period

[1] Unlike Java, values of primitive types like `Int` are also considered objects for the purposes of this discussion.

character), followed by the name of the member, as in `MyModule.abs(-42)`. To use the `toString` member on the object `42`, we'd use `42.toString`. The implementations of members within an object can refer to each other unqualified (without prefixing the object name), but if needed they have access to their enclosing object using a special name: `this`.[2]

Note that even an expression like `2 + 1` is just calling a member of an object. In that case, what we're calling is the `+` member of the object `2`. It's really syntactic sugar for the expression `2.+(1)`, which passes `1` as an argument to the method `+` on the object `2`. Scala has no special notion of *operators*. It's simply the case that `+` is a valid method name in Scala. Any method name can be used infix like that (omitting the dot and parentheses) when calling it with a single argument. For example, instead of `MyModule.abs(42)` we can say `MyModule abs 42` and get the same result. You can use whichever you find more pleasing in any given case.

We can bring an object's member into scope by *importing* it, which allows us to call it unqualified from then on:

```
scala> import MyModule.abs
import MyModule.abs

scala> abs(-42)
res0: 42
```

We can bring *all* of an object's (nonprivate) members into scope by using the under score syntax:

```
import MyModule._
```

2.4 Higher-order functions: passing functions to functions

Now that we've covered the basics of Scala's syntax, we'll move on to covering some of the basics of writing functional programs. The first new idea is this: *functions are values*. And just like values of other types—such as integers, strings, and lists—func tions can be assigned to variables, stored in data structures, and passed as arguments to functions.

When writing purely functional programs, we'll often find it useful to write a func tion that accepts other functions as arguments. This is called a *higher-order function (HOF)*, and we'll look next at some simple examples to illustrate. In later chapters, we'll see how useful this capability really is, and how it permeates the functional pro gramming style. But to start, suppose we wanted to adapt our program to print out both the absolute value of a number *and* the factorial of another number. Here's a sample run of such a program:

```
The absolute value of -42 is 42
The factorial of 7 is 5040
```

[2] Note that in this book, we'll use the term *function* to refer more generally to either so-called standalone functions like `sqrt` or `abs`, or members of some class, including methods. When it's clear from the context, we'll also use the terms *method* and *function* interchangeably, since what matters is not the syntax of invocation (`obj.method(12)` vs. `method(obj, 12)`, but the fact that we're talking about some parameterized block of code.

2.4.1 A short detour: writing loops functionally

First, let's write `factorial`:

```
def factorial(n: Int): Int = { def go(n: Int, acc: Int): Int = if (n <= 0) acc
      else go(n-1, n*acc)

    go(n, 1)
  }
```

An inner function, or local definition. It's common in Scala to write functions that are local to the body of another function. In functional programming, we shouldn't consider this a bigger deal than local integers or strings.

The way we write loops functionally, without mutating a loop variable, is with a recur sive function. Here we're defining a recursive helper function inside the body of the `factorial` function. Such a helper function is often called `go` or `loop` by convention. In Scala, we can define functions inside any block, including within another function definition. Since it's local, the `go` function can only be referred to from within the body of the `factorial` function, just like a local variable would. The definition of `factorial` finally just consists of a call to `go` with the initial conditions for the loop.

The arguments to `go` are the state for the loop. In this case, they're the remaining value `n`, and the current accumulated factorial `acc`. To advance to the next iteration, we simply call `go` recursively with the new loop state (here, `go(n-1, n*acc)`), and to exit from the loop, we return a value without a recursive call (here, we return `acc` in the case that `n <= 0`). Scala detects this sort of *self-recursion* and compiles it to the same sort of bytecode as would be emitted for a `while` loop,[3] so long as the recursive call is in *tail position*. See the sidebar for the technical details on this, but the basic idea is that this optimization[4] (called *tail call elimination*) is applied when there's no addi tional work left to do after the recursive call returns.

Tail calls in Scala

A call is said to be in *tail position* if the caller does nothing other than return the value of the recursive call. For example, the recursive call to `go(n-1,n*acc)` we discussed earlier is in tail position, since the method returns the value of this recursive call directly and does nothing else with it. On the other hand, if we said `1 + go(n-1,n*acc)`, `go` would no longer be in tail position, since the method would still have work to do when `go` returned its result (namely, adding `1` to it).

If all recursive calls made by a function are in tail position, Scala automatically com piles the recursion to iterative loops that don't consume call stack frames for each iteration. By default, Scala doesn't tell us if tail call elimination was successful, but if we're expecting

this to occur for a recursive function we write, we can tell the Scala

[3]We can write `while` loops by hand in Scala, but it's rarely necessary and considered bad form since it hinders good compositional style. [4]The term *optimization* is not really appropriate here. An optimization usually connotes some nonessential per formance improvement, but when we use tail calls to write loops, we generally rely on their being compiled as iterative loops that don't consume a call stack frame for each iteration (which would result in a `StackOverflowError` for large inputs).

compiler about this assumption using the `tailrec` annotation (http://mng.bz/ bWT5), so it can give us a compile error if it's unable to eliminate the tail calls of the function. Here's the syntax for this:

```
def factorial(n: Int): Int = {
  @annotation.tailrec
  def go(n: Int, acc: Int): Int =
    if (n <= 0) acc
    else go(n-1, n*acc)
  go(n, 1)
}
```

We won't talk much more about annotations in this book (you'll find more information at http://mng.bz/GK8T), but we'll use `@annotation.tailrec` extensively.

NOTE See the preface for information on the exercises.

EXERCISE 2.1

Write a recursive function to get the *n*th Fibonacci number (http://mng.bz/C29s). The first two Fibonacci numbers are `0` and `1`. The *n*th number is always the sum of the previous two—the sequence begins `0, 1, 1, 2, 3, 5`. Your definition should use a local tail-recursive function.

```
def fib(n: Int): Int
```

2.4.2 Writing our first higher-order function

Now that we have `factorial`, let's edit our program from before to include it.

```
object MyModule {
  ...
private def formatAbs(x: Int) = {
val msg = "The absolute value of %d is %d." msg.format(x, abs(x))
}

private def formatFactorial(n: Int) = { val msg = "The factorial of %d is %d." msg.format(n,
factorial(n))
}

def main(args: Array[String]): Unit = { println(formatAbs(-42))
println(formatFactorial(7))
```

```
}
}
```

Definitions of `abs` and `factorial` go here.

The two functions, `formatAbs` and `formatFactorial`, are almost identical. If we like, we can generalize these to a single function, `formatResult`, which accepts as an argu ment the *function* to apply to its argument:

```
def formatResult(name: String, n: Int, f: Int => Int) = { val msg = "The %s of %d is %d."
msg.format(name, n, f(n))
}
```

`f` is required to be a function from `Int` to `Int`.

Our `formatResult` function is a higher-order function (HOF) that takes another func tion, called `f` (see sidebar on variable-naming conventions). We give a type to `f`, as we would for any other parameter. Its type is `Int => Int` (pronounced "int to int" or "int arrow int"), which indicates that `f` expects an integer argument and will also return an integer.

Our function `abs` from before matches that type. It accepts an `Int` and returns an `Int`. And likewise, `factorial` accepts an `Int` and returns an `Int`, which also matches the `Int => Int` type. We can therefore pass `abs` or `factorial` as the `f` argument to `formatResult`:

```
scala> formatResult("absolute value", -42, abs)
res0: String = "The absolute value of -42 is 42."

scala> formatResult("factorial", 7, factorial)
res1: String = "The factorial of 7 is 5040."
```

Variable-naming conventions

It's a common convention to use names like `f`, `g`, and `h` for parameters to a higher order function. In functional programming, we tend to use very short variable names, even one-letter names. This is usually because HOFs are so general that they have no opinion on what the argument should actually do. All they know about the argu ment is its type. Many functional programmers feel that short names make code eas ier to read, since it makes the structure of the code easier to see at a glance.

2.5 Polymorphic functions: abstracting over types

So far we've defined only *monomorphic* functions, or functions that operate on only one type of data. For example, `abs` and `factorial` are specific to arguments of type `Int`, and the higher-order function `formatResult` is also fixed to operate on functions that take arguments of type `Int`. Often, and especially when writing HOFs, we want to write code that works for *any* type it's given. These are called *polymorphic functions*,[5] and in the chapters ahead, you'll get plenty of experience writing such functions. Here we'll just introduce the idea.

[5] We're using the term *polymorphism* in a slightly different way than you might be used to if you're familiar with object-oriented programming, where that term usually connotes some form of subtyping or inheritance rela tionship. There are no interfaces or subtyping here in this example. The kind of polymorphism we're using here is sometimes called *parametric polymorphism*.

2.5.1 An example of a polymorphic function

We can often discover polymorphic functions by observing that several monomorphic functions all share a similar structure. For example, the following monomorphic func tion, `findFirst`, returns the first index in an array where the key occurs, or `-1` if it's not found. It's specialized for searching for a `String` in an `Array` of `String` values.

Otherwise, increment `n` and keep looking.

```
def findFirst(ss: Array[String], key: String): Int = { @annotation.tailrec
def loop(n: Int): Int =
if (n >= ss.length) -1
```

If `n` is past the end of the array, return `-1`, indicating the key doesn't exist in the array.

```
    else if (ss(n) == key) n
    else loop(n + 1)

  loop(0)
}
```

Start the loop at the first element of the array.

`ss(n)` extracts the nth element of the array `ss`. If the element at n is equal to the key, return `n`, indicating that the element appears in the array at that index.

The details of the code aren't too important here. What's important is that the code for `findFirst` will look almost identical if we're searching for a `String` in an `Array[String]`, an `Int` in an `Array[Int]`, or an `A` in an `Array[A]` for any given type `A`. We can write `findFirst` more generally for any type `A` by accepting a function to use for testing a particular `A` value.

```
def findFirst[A](as: Array[A], p: A => Boolean): Int = {
@annotation.tailrec
def loop(n: Int): Int = if (n >= as.length) -1 else if (p(as(n))) n else loop(n + 1)

loop(0)
}
```

Instead of hardcoding `String`, take a type `A` as a parameter. And instead of hardcoding an equality check for a given key, take a function with which to test each element of the array.

If the function `p` matches the current element, we've found a match and we return its index in the array.

This is an example of a polymorphic function, sometimes called a *generic* function. We're *abstracting over the type* of the array and the function used for searching it. To write a polymorphic function as a method, we introduce a comma-separated list of *type parameters*, surrounded by square brackets (here, just a single `[A]`), following the name of the function, in this case `findFirst`. We can call the type parameters any thing we want—`[Foo, Bar, Baz]` and `[TheParameter, another_good_one]` are valid type parameter declarations—though by convention we typically use short, one-letter, uppercase type parameter names like `[A,B,C]`.

The type parameter list introduces *type variables* that can be referenced in the rest of the type signature (exactly analogous to how variables introduced in the parameter list to a function can be referenced in the body of the function). In `findFirst`, the type variable `A` is referenced in two places: the elements of the array are required to have the type `A` (since it's an `Array[A]`), and the `p` function must accept a value of type `A` (since it's a function of type `A => Boolean`). The fact that the same type variable is referenced in both places in the type signature implies that the type must be the same for both arguments, and the compiler will enforce this fact anywhere we try to call `findFirst`. If we try to search for a `String` in an `Array[Int]`, for instance, we'll get a type mismatch error.



EXERCISE 2.2

Implement `isSorted`, which checks whether an `Array[A]` is sorted according to a given comparison function:

```
def isSorted[A](as: Array[A], ordered: (A,A) => Boolean): Boolean
```

2.5.2 Calling HOFs with anonymous functions

When using HOFs, it's often convenient to be able to call these functions with *anony mous functions* or *function literals*, rather than having to supply some existing named function. For instance, we can test the `findFirst` function in the REPL as follows:

```
scala> findFirst(Array(7, 9, 13), (x: Int) => x == 9)
res2: Int = 1
```

There is some new syntax here. The expression `Array(7, 9, 13)` is an *array literal*. It constructs a new array with three integers in it. Note the lack of a keyword like `new` to construct the array.

The syntax `(x: Int) => x == 9` is a *function literal* or *anonymous function*. Instead of defining this function as a method with a name, we can define it inline using this con venient syntax. This particular function takes one argument called `x` of type `Int`, and it returns a `Boolean` indicating whether `x` is equal to `9`.

In general, the arguments to the function are declared to the left of the `=>` arrow, and we can then use them in the body of the function to the right of the arrow. For example, if we want to write an equality function that takes two integers and checks if they're equal to each other, we could write that like this:

```
scala> (x: Int, y: Int) => x == y
res3: (Int, Int) => Boolean = <function2>
```

The `<function2>` notation given by the REPL indicates that the value of `res3` is a func tion that takes two arguments. When the type of the function's inputs can be inferred

by Scala from the context, the type annotations on the function's arguments may be elided, for example, `(x,y) => x < y`. We'll see an example of this in the next section, and lots more examples throughout this book.

Functions as values in Scala

When we define a function literal, what is actually being defined in Scala is an object with a method called `apply`. Scala has a special rule for this method name, so that objects that have an `apply` method can be called as if they were themselves meth ods. When we define a function literal like `(a, b) => a < b`, this is really syntactic sugar for object creation:

```
val lessThan = new Function2[Int, Int, Boolean] {
  def apply(a: Int, b: Int) = a < b
}
```

`lessThan` has type `Function2[Int,Int,Boolean]`, which is usually written `(Int,Int) => Boolean`. Note that the `Function2` interface (known in Scala as a *trait*) has an `apply` method. And when we call the `lessThan` function with `lessThan(10, 20)`, it's really syntactic sugar for calling its `apply` method:

```
scala> val b = lessThan.apply(10, 20)
b: Boolean = true
```

`Function2` is just an ordinary trait (an interface) provided by the standard Scala library (API docs link: http://mng.bz/qFMr) to represent function objects that take two arguments. Also provided are `Function1`, `Function3`, and others, taking a num ber of arguments indicated by the name. Because functions are just ordinary Scala objects, we say that they're *first-class* values. We'll often use "function" to refer to either such a first-class function or a method, depending on context.

2.6 Following types to implementations

As you might have seen when writing `isSorted`, the universe of possible implementa tions is significantly reduced when implementing a polymorphic function. If a func tion is polymorphic in some type `A`, the only operations that can be performed on that `A` are those passed into the function as arguments (or that can be defined in terms of these given operations).[6] In some cases, you'll find that the universe of possibilities for a given polymorphic type is constrained such that only one implementation is possible!

Let's look at an example of a function signature that can only be implemented in one way. It's a higher-order function for performing what's called *partial application*. This function, `partial1`, takes a value and a function of two arguments, and returns a function of one argument as its result. The name comes from the fact that the func tion is being applied to some but not all of the arguments it requires:

```
def partial1[A,B,C](a: A, f: (A,B) => C): B => C
```

[6] Technically, all values in Scala can be compared for equality (using ==), and turned into strings with `toString` and integers with `hashCode`. But this is something of a wart inherited from Java.

The `partial1` function has three type parameters: `A`, `B`, and `C`. It then takes two argu ments. The argument `f` is itself a function that takes two arguments of types `A` and `B`, respectively, and returns a value of type `C`. The value returned by `partial1` will also be a function, of type `B => C`.

How would we go about implementing this higher-order function? It turns out that there's only one implementation that compiles, and it follows logically from the type signature. It's like a fun little logic puzzle.[7]

Let's start by looking at the type of thing that we have to return. The return type of `partial1` is `B => C`, so we know that we have to return a function of that type. We can just begin writing a function literal that takes an argument of type `B`:

```
def partial1[A,B,C](a: A, f: (A,B) => C): B => C =
  (b: B) => ???
```

This can be weird at first if you're not used to writing anonymous functions. Where did that `B` come from? Well, we've just written, "Return a function that takes a value `b` of type `B`." On the right-hand-side of the `=>` arrow (where the question marks are now) comes the body of that anonymous function. We're free to refer to the value `b` in there for the same reason that we're allowed to refer to the value `a` in the body of `partial1`.[8]

Let's keep going. Now that we've asked for a value of type `B`, what do we want to return from our anonymous function? The type signature says that it has to be a value of type `C`. And there's only one way to get such a value. According to the signature, `C` is the return type of the function `f`. So the only way to get that `C` is to pass an `A` and a `B` to `f`. That's easy:

```
def partial1[A,B,C](a: A, f: (A,B) => C): B => C =
  (b: B) => f(a, b)
```

And we're done! The result is a higher-order function that takes a function of two arguments and partially applies it. That is, if we have an `A` and a function that needs both `A` and `B` to produce `C`, we can get a function that just needs `B` to produce `C` (since we already have the `A`). It's like saying, "If I can give you a carrot for an apple and a banana, and you already gave me an apple, you just have to give me a banana and I'll give you a carrot."

Note that the type annotation on `b` isn't needed here. Since we told Scala the return type would be `B => C`, Scala knows the type of `b` from the context and we could just write `b => f(a,b)` as the implementation. Generally speaking, we'll omit the type annotation on a function literal if it can be inferred by Scala.

[7] Even though it's a fun puzzle, this isn't a purely academic exercise. Functional programming in practice involves a lot of fitting building blocks together in the only way that makes sense. The purpose of this exercise is to get practice using higher-order functions, and using Scala's type system to guide your programming. [8] Within the body of this inner function, the outer `a` is still in scope. We sometimes say that the inner function *closes over* its environment, which includes `a`.



EXERCISE 2.3

Let's look at another example, *currying*,[9] which converts a function `f` of two arguments into a function of one argument that partially applies `f`. Here again there's only one implementation that compiles. Write this implementation.

```
def curry[A,B,C](f: (A, B) => C): A => (B => C)
```

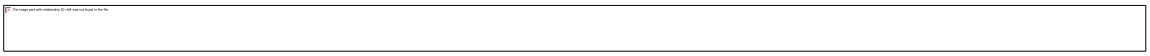


EXERCISE 2.4

Implement `uncurry`, which reverses the transformation of `curry`. Note that since `=>` associates to the right, `A => (B => C)` can be written as `A => B => C`.

```
def uncurry[A,B,C](f: A => B => C): (A, B) => C
```

Let's look at a final example, *function composition*, which feeds the output of one func tion to the input of another function. Again, the implementation of this function is fully determined by its type signature.



EXERCISE 2.5 Implement the higher-order function that composes two functions.

```
def compose[A,B,C](f: B => C, g: A => B): A => C
```

This is such a common thing to want to do that Scala's standard library provides `compose` as a method on `Function1` (the interface for functions that take one argu ment). To compose two functions `f` and `g`, we simply say `f compose g`.[10] It also pro vides an `andThen` method. `f andThen g` is the same as `g compose f`:

```
scala> val f = (x: Double) => math.Pi/2-x
f: Double => Double = <function1>

scala> val cos = f andThen math.sin
cos: Double => Double = <function1>
```

It's all well and good to puzzle together little one-liners like this, but what about pro gramming with a large real-world code base? In functional programming, it turns out to be exactly the same. Higher-order functions like `compose` don't care whether

[9] This is named after the mathematician Haskell Curry, who discovered the principle. It was independently dis covered earlier by Moses Schoenfinkel, but *Schoenfinkelization* didn't catch on. [10] Solving the `compose` exercise by using this library function is considered cheating.

they're operating on huge functions backed by millions of lines of code or functions that are simple one-liners. Polymorphic, higher-order functions often end up being extremely widely applicable, precisely because they say nothing about any particular domain and are simply abstracting over a common pattern that occurs in many con texts. For this reason, programming in the large has much the same flavor as program ming in the small. We'll write a lot of widely applicable functions over the course of this book, and the exercises in this chapter are a taste of the style of reasoning you'll employ when writing such functions.

2.7 Summary

In this chapter, we learned enough of the Scala language to get going, and some pre liminary functional programming concepts. We learned how to define simple func tions and programs, including how we can express loops using recursion; then we introduced the idea of higher-order functions, and we got some practice writing poly morphic functions in Scala. We saw how the implementations of polymorphic func tions are often significantly constrained, such that we can often simply "follow the types" to the correct implementation. This is something we'll see a lot of in the chap ters ahead.

Although we haven't yet written any large or complex programs, the principles we've discussed here are scalable and apply equally well to programming in the large as they do to programming in the small.

Next we'll look at using pure functions to manipulate data.

Functional data structures

We said in the introduction that functional programs don't update variables or modify mutable data structures. This raises pressing questions: what sort of data structures *can* we use in functional programming, how do we define them in Scala, and how do we operate on them? In this chapter, we'll learn the concept of *func tional data structures* and how to work with them. We'll use this as an opportunity to introduce how data types are defined in functional programming, learn about the related technique of *pattern matching*, and get practice writing and generalizing pure functions.

This chapter has a lot of exercises, particularly to help with this last point—writ ing and generalizing pure functions. Some of these exercises may be challenging. As always, consult the hints or the answers at our GitHub site (https://github.com/fpinscala/fpinscala; see the preface), or ask for help online if you need to.

3.1 Defining functional data structures

A functional data structure is (not surprisingly) operated on using only pure func tions. Remember, a pure function must not change data in place or perform other side effects. *Therefore, functional data structures are by definition immutable.* For exam ple, the

empty list (written `List()` or `Nil` in Scala) is as eternal and immutable as the integer values `3` or `4`. And just as evaluating `3 + 4` results in a new number `7` with out modifying either `3` or `4`, concatenating two lists together (the syntax for this is `a ++ b` for two lists `a` and `b`) yields a new list and leaves the two inputs unmodified.

Doesn't this mean we end up doing a lot of extra copying of the data? Perhaps surprisingly, the answer is no, and we'll talk about exactly why that is. But first let's examine what's probably the most ubiquitous functional data structure, the singly linked list. The definition here is identical in spirit to (though simpler than) the

`List` data type defined in Scala's standard library. This code listing introduces a lot of new syntax and concepts that we'll talk through in detail.

`List` companion object. Contains functions for creating and working with lists.

```
package fpinscala.datastructures


sealed trait List[+A]
case object Nil extends List[Nothing]
```

`List` data type, parameterized on a type, `A`. A `List` data constructor representing the empty list.

```


case class Cons[+A](head: A, tail: List[A]) extends List[A]


object List {
def sum(ints: List[Int]): Int = ints match {
case Nil => 0
case Cons(x,xs) => x + sum(xs)
}
```

The sum of a list starting with `x` is `x` plus the sum of the rest of the list.

The sum of the empty list is 0.

```
 def product(ds: List[Double]): Double = ds match { case Nil => 1.0
case Cons(0.0, _) => 0.0
case Cons(x,xs) => x * product(xs)
}
```

Another data constructor, representing nonempty lists. Note that `tail` is another `List[A]`, which may be `Nil` or another `Cons`.

A function that uses pattern matching to add up a list of integers.

```
def apply[A](as: A*): List[A] =
if (as.isEmpty) Nil
else Cons(as.head, apply(as.tail: _*)) }
```

Variadic function syntax.

Let's look first at the definition of the data type, which begins with the keywords `sealed trait`. In general, we introduce a data type with the `trait` keyword. A `trait` is an abstract interface that may optionally contain implementations of some meth ods. Here we're declaring a `trait`, called `List`, with no methods on it. Adding `sealed` in front means that all implementations of the `trait` must be declared in this file.[1]

There are two such implementations, or *data constructors*, of `List` (each introduced with the keyword `case`) declared next, to represent the two possible forms a `List` can take. As the figure at the top of page 31 shows, a `List` can be empty, denoted by the data constructor `Nil`, or it can be nonempty, denoted by the data constructor `Cons` (traditionally short for *construct*). A nonempty list consists of an initial element, `head`, followed by a `List` (possibly empty) of remaining elements (the `tail`):

```
case object Nil extends List[Nothing]
case class Cons[+A](head: A, tail: List[A]) extends List[A]
```

[1] We could also say `abstract class` here instead of `trait`. The distinction between the two is not at all sig nificant for our purposes right now. See section 5.3 in the Scala Language Specification (http://mng.bz/ R75t) for more on the distinction.

Singly linked lists

`List("a", "b")`

`Cons("a", Cons("b", Nil))`

These mean the same thing.

They mean this data structure in memory.

Cons

`"a"`

Head Tail

Cons

`"b"`

Head Tail

Nil

Just as functions can be polymorphic, data types can be as well, and by adding the type parameter `[+A]` after `sealed trait List` and

then using that `A` parameter inside of the `Cons` data constructor, we declare the `List` data type to be polymorphic in the type of elements it contains, which means we can use this same definition for a list of `Int` elements (denoted `List[Int]`), `Double` elements (denoted `List[Double]`), `String` elements (`List[String]`), and so on (the `+` indicates that the type parameter `A` is *covariant*—see sidebar "More about variance" for more information).

A data constructor declaration gives us a function to construct that form of the data type. Here are a few examples:

```
val ex1: List[Double] = Nil
val ex2: List[Int] = Cons(1, Nil)
val ex3: List[String] = Cons("a", Cons("b", Nil))
```

The `case object Nil` lets us write `Nil` to construct an empty `List`, and the `case class Cons` lets us write `Cons(1, Nil)`, `Cons("a", Cons("b", Nil))` and so on to build singly linked lists of arbitrary lengths.[2] Note that because `List` is parameterized on a type, `A`, these are polymorphic functions that can be instantiated with different types for `A`. Here, `ex2` instantiates the `A` type parameter to `Int`, while `ex3` instantiates it to `String`. The `ex1` example is interesting—`Nil` is being instantiated with type `List[Double]`, which is allowed because the empty list contains no elements and can be considered a list of whatever type we want!

Each data constructor also introduces a *pattern* that can be used for *pattern matching*, as in the functions `sum` and `product`. We'll examine pattern matching in more detail next.

More about variance

In the declaration `trait List[+A]`, the `+` in front of the type parameter `A` is a *variance annotation* that signals that `A` is a *covariant* or "positive" parameter of `List`. This means that, for instance, `List[Dog]` is considered a subtype of `List[Animal]`, assuming `Dog` is a subtype of `Animal`. (More generally, for all types `X` and `Y`, if `X` is a

[2] Scala generates a default `def toString: String` method for any `case class` or `case object`, which can be convenient for debugging. You can see the output of this default `toString` implementation if you experiment with `List` values in the REPL, which uses `toString` to render the result of each expression. `Cons(1,Nil)` will be printed as the string `"Cons(1, Nil)"`, for instance. But note that the generated `toString` will be naively recursive and will cause stack overflow when printing long lists, so you may wish to provide a different implementation.

(continued)

subtype of `Y`*, then* `List[X]` *is a subtype of* `List[Y]`*). We could leave out the* `+` *in front of the* `A`*, which would make* `List` *invariant in that type parameter.*

But notice now that `Nil` *extends* `List[Nothing]`*.* `Nothing` *is a subtype of all types, which means that in conjunction with the variance annotation,* `Nil` *can be considered a* `List[Int]`*, a* `List[Double]`*, and so on, exactly as we want.*

These concerns about variance aren't very important for the present discussion and are more of an artifact of how Scala encodes data constructors via subtyping, so don't worry if this is not completely clear right now. It's certainly possible to write code without using variance annotations at all, and function signatures are some times simpler (whereas type inference often gets worse). We'll use variance annota tions throughout this book where it's convenient to do so, but you should feel free to experiment with both approaches.

3.2 Pattern matching

Let's look in detail at the functions `sum` and `product`, which we place in the `object List`, sometimes called the *companion object* to `List` (see sidebar). Both these defini tions make use of pattern matching:

```
def sum(ints: List[Int]): Int = ints match {
  case Nil => 0
  case Cons(x,xs) => x + sum(xs)
}

def product(ds: List[Double]): Double = ds match {
  case Nil => 1.0
  case Cons(0.0, _) => 0.0
  case Cons(x,xs) => x * product(xs)
}
```

As you might expect, the `sum` function states that the sum of an empty list is `0`, and the sum of a nonempty list is the first element, `x`, plus the sum of the remaining elements, `xs`.[3] Likewise the `product` definition states that the product of an empty list is `1.0`, the product of any list starting with `0.0` is `0.0`, and the product of any other nonempty list is the first element multiplied by the product of the remaining elements. Note that these are recursive definitions, which are common when writing functions that oper ate over recursive data types like `List` (which refers to itself recursively in its `Cons` data constructor).

Pattern matching works a bit like a fancy `switch` statement that may descend into the structure of the expression it examines and extract subexpressions of that

[3] We could call `x` and `xs` anything there, but it's a common convention to use `xs`, `ys`, `as`, or `bs` as variable names for a sequence of some sort, and `x`, `y`, `z`, `a`, or `b` as the name for a single element of a sequence. Another common naming convention is `h` for the first element of a list (the *head* of the list), `t` for the remaining ele ments (the *tail*), and `l` for an entire list.

structure. It's introduced with an expression (the *target* or *scrutinee*) like `ds`, followed by the keyword `match`, and a `{}`-wrapped sequence of *cases*. Each case in the match consists of a *pattern* (like `Cons(x,xs)`) to the left of the `=>` and a *result* (like `x * product(xs)`) to the right of the `=>`. If the target *matches* the pattern in a case (discussed next), the result of that case becomes the result of the entire match expression. If multiple pat terns match the target, Scala chooses the first matching case.

Companion objects in Scala

We'll often declare a companion object in addition to our data type and its data con structors. This is just an `object` *with the same name as the data type (in this case* `List`*) where we put various convenience functions for creating or working with values of the data type.*

If, for instance, we wanted a function `def fill[A](n: Int, a: A): List[A]` *that created a* `List` *with* `n` *copies of the element* `a`*, the* `List` *companion object would be a good place for it. Companion objects are more of a convention in Scala.*[4] *We could have called this module* `Foo` *if we wanted, but calling it* `List` *makes it clear that the module contains functions relevant to working with lists.*

Let's look at a few more examples of pattern matching:

- `List(1,2,3) match { case _ => 42 }` results in `42`. Here we're using a variable pattern, `_`, which matches any expression. We could say `x` or `foo` instead of `_`, but we usually use `_` to indicate a variable whose value we ignore in the result of the case.[5]
- `List(1,2,3) match { case Cons(h,_) => h }` results in `1`. Here we're using a data constructor pattern in conjunction with variables to *capture* or *bind* a subex pression of the target.
- `List(1,2,3) match { case Cons(_,t) => t }` results in `List(2,3)`.
- `List(1,2,3) match { case Nil => 42 }` results in a `MatchError` at runtime. A `MatchError` indicates that none of the cases in a match expression matched the target.

Matching a list

`Cons(1, Cons(2, Cons(3, Nil)))`

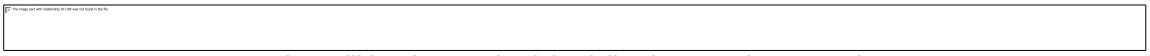
Equal

`List(1, 2, 3) match {case Cons(h, _) => h}`

The result is 1.

[4] There is some special support for them in the language that isn't really relevant for our purposes. [5] The `_` variable pattern is treated somewhat specially in that it may be mentioned multiple times in the pattern to ignore multiple parts of the target.

What determines if a pattern matches an expression? A pattern may contain *literals* like `3` or `"hi"`; *variables* like `x` and `xs`, which match anything, indicated by an identifier start ing with a lowercase letter or underscore; and data constructors like `Cons(x,xs)` and `Nil`, which match only values of the corresponding form. (`Nil` as a pattern matches only the value `Nil`, and `Cons(h,t)` or `Cons(x,xs)` as a pattern only match `Cons` values.) These components of a pattern may be nested arbitrarily—`Cons(x1, Cons(x2, Nil))` and `Cons(y1, Cons(y2, Cons(y3, _)))` are valid patterns. A pattern *matches* the target if there exists an assignment of variables in the pattern to subexpressions of the target that make it *structurally equivalent* to the target. The resulting expression for a matching case will then have access to these variable assignments in its local scope.



EXERCISE 3.1 What will be the result of the following match expression?

```
val x = List(1,2,3,4,5) match {
  case Cons(x, Cons(2, Cons(4, _))) => x
  case Nil => 42
  case Cons(x, Cons(y, Cons(3, Cons(4, _)))) =>x+y
  case Cons(h, t) => h + sum(t)
  case _ => 101
}
```

You're strongly encouraged to try experimenting with pattern matching in the REPL to get a sense for how it behaves.

Variadic functions in Scala

The function `apply` in the `object List` is a *variadic function*, meaning it accepts zero or more arguments of type `A`:

```
def apply[A](as: A*): List[A] =
  if (as.isEmpty) Nil
  else Cons(as.head, apply(as.tail: _*))
```

For data types, it's a common idiom to have a variadic `apply` method in the compan ion object to conveniently construct instances of the data type. By calling this function `apply` and placing it in the companion object, we can invoke it with syntax like `List(1,2,3,4)` or `List("hi","bye")`, with as many values as we want separated by commas (we sometimes call this the *list literal* or just *literal* syntax).

Variadic functions are just providing a little syntactic sugar for creating and passing a `Seq` of elements explicitly. `Seq` is the interface in Scala's collections library imple mented by sequence-like data structures such as lists, queues, and vectors. Inside `apply`, the argument `as` will be bound to a `Seq[A]` (documentation at http://mng.bz/ f4k9), which has the functions `head` (returns the first element) and `tail` (returns all elements but the first). The special `_*` type annotation allows

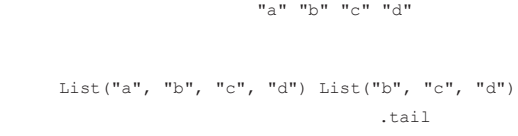
us to pass a `Seq` to a variadic method.

3.3 Data sharing in functional data structures

When data is immutable, how do we write functions that, for example, add or remove elements from a list? The answer is simple. When we add an element `1` to the front of an existing list, say `xs`, we return a new list, in this case `Cons(1,xs)`. Since lists are immutable, we don't need to actually copy `xs`; we can just reuse it. This is called *data sharing*. Sharing of immutable data often lets us implement functions more efficiently; we can always return immutable data structures without having to worry about subse quent code modifying our data. There's no need to pessimistically make copies to avoid modification or corruption.[6]

In the same way, to remove an element from the front of a list `mylist = Cons(x,xs)`, we simply return its tail, `xs`. There's no real removing going on. The orig inal list, `mylist`, is still available, unharmed. We say that functional data structures are *persistent*, meaning that existing references are never changed by operations on the data structure.

Data sharing



Both lists share the same data in memory. `.tail` does not modify the original list, it simply references the tail of the original list. Defensive copying is not needed, because the list is immutable.

Let's try implementing a few different functions for modifying lists in different ways. You can place this, and other functions that we write, inside the `List` companion object.

EXERCISE 3.2

Implement the function `tail` for removing the first element of a `List`. Note that the function takes constant time. What are different choices you could make in your implementation if the `List` is `Nil`? We'll return to this question in the next chapter.

[6] Pessimistic copying can become a problem in large programs. When mutable data is passed through a chain of loosely coupled components, each component has to make its own copy of the data because other compo nents might modify it. Immutable data is always safe to share, so we never have to make copies. We find that *in the large*, FP can often achieve greater efficiency than approaches that rely on side effects, due to much greater sharing of data and computation.

EXERCISE 3.3

Using the same idea, implement the function `setHead` for replacing the first element of a `List` with a different value.

3.3.1 The efficiency of data sharing

Data sharing often lets us implement operations more efficiently. Let's look at a few examples.

EXERCISE 3.4

Generalize `tail` to the function `drop`, which removes the first `n` elements from a list. Note that this function takes time proportional only to the number of elements being dropped—we don't need to make a copy of the entire `List`.

```
def drop[A](l: List[A], n: Int): List[A]
```

EXERCISE 3.5

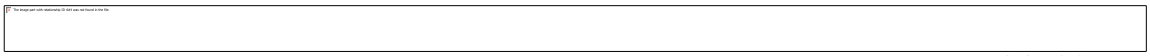
Implement `dropWhile`, which removes elements from the `List` prefix as long as they match a predicate.

```
def dropWhile[A](l: List[A], f: A => Boolean): List[A]
```

A more surprising example of data sharing is this function that adds all the elements of one list to the end of another:

```
def append[A](a1: List[A], a2: List[A]): List[A] =
  a1 match {
    case Nil => a2
    case Cons(h,t) => Cons(h, append(t, a2))
  }
```

Note that this definition only copies values until the first list is exhausted, so its run time and memory usage are determined only by the length of `a1`. The remaining list then just points to `a2`. If we were to implement this same function for two arrays, we'd be forced to copy all the elements in both arrays into the result. In this case, the immutable linked list is much more efficient than an array!



EXERCISE 3.6

Not everything works out so nicely. Implement a function, `init`, that returns a `List` consisting of all but the last element of a `List`. So, given `List(1,2,3,4)`, `init` will return `List(1,2,3)`. Why can't this function be implemented in constant time like `tail`?

```
def init[A](l: List[A]): List[A]
```

Because of the structure of a singly linked list, any time we want to replace the `tail` of a `Cons`, even if it's the last `Cons` in the list, we must copy all the previous `Cons` objects. Writing purely functional data structures that support different operations efficiently is all about finding clever ways to exploit data sharing. We're not going to cover these data structures here; for now, we're content to use the functional data structures oth ers have written. As an example of what's possible, in the Scala standard library there's a purely functional sequence implementation, `Vector` (documentation at http:// mng.bz/Xhl8), with constant-time random access, updates, `head`, `tail`, `init`, and con stant-time additions to either the front or rear of the sequence. See the chapter notes for links to further reading about how to design such data structures.

3.3.2 Improving type inference for higher-order functions

Higher-order functions like `dropWhile` will often be passed anonymous functions. Let's look at a typical example.

Recall the signature of `dropWhile`:

```
def dropWhile[A](l: List[A], f: A => Boolean): List[A]
```

When we call it with an anonymous function for `f`, we have to specify the type of its argument, here named `x`:

```
val xs: List[Int] = List(1,2,3,4,5)
val ex1 = dropWhile(xs, (x: Int) => x < 4)
```

The value of `ex1` is `List(4,5)`.

It's a little unfortunate that we need to state that the type of `x` is `Int`. The first argu ment to `dropWhile` is a `List[Int]`, so the function in the second argument must accept an `Int`. Scala can infer this fact if we group `dropWhile` into two argument lists:

```
def dropWhile[A](as: List[A])(f: A => Boolean): List[A] =
  as match {
    case Cons(h,t) if f(h) => dropWhile(t)(f)
    case _ => as
  }
```


The syntax for calling this version of `dropWhile` looks like `dropWhile(xs)(f)`. That is, `dropWhile(xs)` is returning a function,

which we then call with the argument `f` (in other words, `dropWhile` is curried[7]). The main reason for grouping the arguments this way is to assist with type inference. We can now use `dropWhile` without annotations:

```
val xs: List[Int] = List(1,2,3,4,5)
val ex1 = dropWhile(xs)(x => x < 4)
```

Note that `x` is not annotated with its type.

More generally, when a function definition contains multiple argument groups, type information flows from left to right across these argument groups. Here, the first argument group fixes the type parameter `A` of `dropWhile` to `Int`, so the annotation on `x => x < 4` is not required.[8]

We'll often group and order our function arguments into multiple argument lists to maximize type inference.

3.4 Recursion over lists and generalizing to higher-order functions

Let's look again at the implementations of `sum` and `product`. We've simplified the `product` implementation slightly, so as not to include the "short-circuiting" logic of checking for `0.0`:

```
def sum(ints: List[Int]): Int = ints match {
  case Nil => 0
  case Cons(x,xs) => x + sum(xs)
}

def product(ds: List[Double]): Double = ds match {
  case Nil => 1.0
  case Cons(x, xs) => x * product(xs)
}
```

Note how similar these two definitions are. They're operating on different types (`List[Int]` versus `List[Double]`), but aside from this, the only differences are the value to return in the case that the list is empty (`0` in the case of `sum`, `1.0` in the case of `product`), and the operation to combine results (`+` in the case of `sum`, `*` in the case of `product`). Whenever you encounter duplication like this, you can generalize it away by pulling subexpressions out into function arguments. If a subexpression refers to any local variables (the `+` operation refers to the local variables `x` and `xs` introduced by the pattern, similarly for `product`), turn the subexpression into a function that accepts these variables as arguments. Let's do that now. Our function will take as arguments

[7] Recall from the previous chapter that a function of two arguments can be represented as a function that accepts one argument and returns another function of one argument. [8] This is an unfortunate restriction of the Scala compiler; other functional languages like Haskell and OCaml provide *complete* inference, meaning type annotations are almost never required. See the notes for this chapter for more information and links to further reading.

the value to return in the case of the empty list, and the function to add an element to the result in the case of a nonempty list.[9]

```
def foldRight[A,B](as: List[A], z: B)(f: (A, B) => B): B =
  as match {
case Nil => z
case Cons(x, xs) => f(x, foldRight(xs, z)(f)) }
```

```
def sum2(ns: List[Int]) =
```

Again, placing `f` in its own argument group after `as` and `z` lets type inference determine the input types to `f`.

```
foldRight(ns, 0)((x,y) => x + y)

def product2(ns: List[Double]) = foldRight(ns, 1.0)(_ * _)
```

`_ * _` is more concise notation for `(x,y) => x * y` (see following sidebar).

`foldRight` is not specific to any one type of element, and we discover while generaliz ing that the value that's returned doesn't have to be of the same type as the elements of the list! One way of describing what `foldRight` does is that it replaces the construc tors of the list, `Nil` and `Cons`, with `z` and `f`, illustrated here:

```
Cons(1, Cons(2, Nil))
f (1, f (2, z ))
```

Let's look at a complete example. We'll *trace* the evaluation of `foldRight(Cons(1, Cons(2, Cons(3, Nil))), 0)((x,y) => x + y)` by repeatedly substituting the defi nition of `foldRight` for its usages. We'll use program traces like this throughout this book:

```
foldRight(Cons(1, Cons(2, Cons(3, Nil))),
0)((x,y) => x + y) 1 + foldRight(Cons(2,
Cons(3, Nil)), 0)((x,y) => x + y)
1 + (2 + foldRight(Cons(3, Nil), 0)((x,y) => x + y))
1 + (2 + (3 + (foldRight(Nil, 0)((x,y) => x + y)))) 1 + (2 + (3 + (0)))
6
```

Replace `foldRight` with its definition.

Note that `foldRight` must traverse all the way to the end of the list (pushing frames onto the call stack as it goes) before it can begin collapsing it.

Underscore notation for anonymous functions

The anonymous function `(x,y) => x + y` can be written as `_ + _` in situations where the types of `x` and `y` could be inferred by Scala. This is a useful shorthand in cases where the function parameters are mentioned just once in the body of the function. Each underscore in an anonymous function expression like `_ + _` introduces a new (unnamed) function parameter and references it. Arguments are introduced in left-to right order. Here are a few more examples:

[9] In the Scala standard library, `foldRight` is a method on `List` and its arguments are curried similarly for bet ter type inference.


```
_+_

_*2

_.head

_ drop _
```

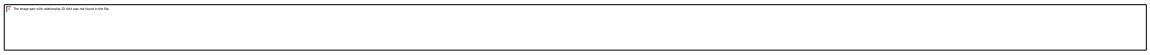
`(x,y) => x + y`

`x => x * 2`

```
xs => xs.head

(xs,n) => xs.drop(n)
```

Use this syntax judiciously. The meaning of this syntax in expressions like `foo(_, g(List(_ + 1), _))` *can be unclear. There are precise rules about scoping of these underscore-based anonymous functions in the Scala language specification, but if you have to think about it, we recommend just using ordinary named function parameters.*



EXERCISE 3.7

Can `product`, implemented using `foldRight`, immediately halt the recursion and return `0.0` if it encounters a `0.0`? Why or why not? Consider how any short-circuiting might work if you call `foldRight` with a large list. This is a deeper question that we'll return to in chapter 5.



EXERCISE 3.8

See what happens when you pass `Nil` and `Cons` themselves to `foldRight`, like this: `foldRight(List(1,2,3), Nil:List[Int])(Cons(_,_))`.[10] What do you think this says about the relationship between `foldRight` and the data constructors of `List`?

EXERCISE 3.9 Compute the length of a list using `foldRight`.

```
def length[A](as: List[A]): Int
```

EXERCISE 3.10

Our implementation of `foldRight` is not tail-recursive and will result in a `StackOver flowError` for large lists (we say it's not *stack-safe*). Convince yourself that this is the case, and then write another general list-recursion function, `foldLeft`, that is

[10] The type annotation `Nil:List[Int]` is needed here, because otherwise Scala infers the `B` type parameter in `foldRight` as `List[Nothing]`.

tail-recursive, using the techniques we discussed in the previous chapter. Here is its signature:[11]

```
def foldLeft[A,B](as: List[A], z: B)(f: (B, A) => B): B
```

EXERCISE 3.11 Write `sum`, `product`, and a function to compute the length of a list using `foldLeft`.

EXERCISE 3.12

Write a function that returns the reverse of a list (given `List(1,2,3)` it returns `List(3,2,1)`). See if you can write it using a fold.

EXERCISE 3.13

Hard: Can you write `foldLeft` in terms of `foldRight`? How about the other way around? Implementing `foldRight` via `foldLeft` is useful because it lets us implement `foldRight` tail-recursively, which means it works even for large lists without overflow ing the stack.

EXERCISE 3.14 Implement `append` in terms of either `foldLeft` or `foldRight`.

EXERCISE 3.15

Hard: Write a function that concatenates a list of lists into a single list. Its runtime should be linear in the total length of all lists. Try to use functions we have already defined.

3.4.1 More functions for working with lists

There are many more useful functions for working with lists. We'll cover a few more here, to get additional practice with generalizing functions and to get some basic famil iarity with common patterns when processing lists. After finishing this section, you're not going to emerge with an automatic sense of when to use each of these functions.

[11] Again, `foldLeft` is defined as a method of `List` in the Scala standard library, and it is curried similarly for better type inference, so you can write `mylist.foldLeft(0.0)(_ + _)`.

Just get in the habit of looking for possible ways to generalize any explicit recursive functions you write to process lists. If you do this, you'll (re)discover these functions for yourself and develop an instinct for when you'd use each one.

EXERCISE 3.16

Write a function that transforms a list of integers by adding `1` to each element. (Reminder: this should be a pure function that returns a new `List`!)

EXERCISE 3.17

Write a function that turns each value in a `List[Double]` into a `String`. You can use the expression `d.toString` to convert some `d: Double` to a `String`.

EXERCISE 3.18

Write a function `map` that generalizes modifying each element in a list while maintain ing the structure of the list. Here is its signature:[12]

```
def map[A,B](as: List[A])(f: A => B): List[B]
```

EXERCISE 3.19

Write a function `filter` that removes elements from a list unless they satisfy a given predicate. Use it to remove all odd numbers from a `List[Int]`.

```
def filter[A](as: List[A])(f: A => Boolean): List[A]
```

EXERCISE 3.20

Write a function `flatMap` that works like `map` except that the function given will return a list instead of a single result, and that list should be inserted into the final resulting list. Here is its signature:

```
def flatMap[A,B](as: List[A])(f: A => List[B]): List[B]
```

For instance, `flatMap(List(1,2,3))(i => List(i,i))` should result in `List(1,1,2,2,3,3)`.

[12] In the standard library, `map` and `flatMap` are methods of `List`.

EXERCISE 3.21 Use `flatMap` to implement `filter`.

EXERCISE 3.22

Write a function that accepts two lists and constructs a new list by adding correspond ing elements. For example, `List(1,2,3)` and `List(4,5,6)` become `List(5,7,9)`.

EXERCISE 3.23

Generalize the function you just wrote so that it's not specific to integers or addition. Name your generalized function `zipWith`.

LISTS IN THE STANDARD LIBRARY

`List` exists in the Scala standard library (API documentation at http://mng.bz/vu45), and we'll use the standard library version in subsequent chapters. The main difference between the `List` developed here and the standard library version is that `Cons` is called `::`, which associates to the right,[13] so `1 :: 2 :: Nil` is equal to `1 :: (2 :: Nil)`, which is equal to `List(1,2)`. When pattern matching, `case Cons(h,t)` becomes `case h :: t`, which avoids having to nest parentheses if writing a pattern like `case h :: h2 :: t` to extract more than just the first element of the `List`.

There are a number of other useful methods on the standard library lists. You may want to try experimenting with these and other methods in the REPL after reading the API documentation. These are defined as methods on `List[A]`, rather than as stand alone functions as we've done in this chapter:

- `def take(n: Int): List[A]`—Returns a list consisting of the first `n` elements of `this`
- `def takeWhile(f: A => Boolean): List[A]`—Returns a list consisting of the lon gest valid prefix of `this` whose elements all pass the predicate `f`
- `def forall(f: A => Boolean): Boolean`—Returns `true` if and only if all ele ments of `this` pass the predicate `f`
- `def exists(f: A => Boolean): Boolean`—Returns `true` if any element of `this` passes the predicate `f`

[13] In Scala, all methods whose names end in `:` are right-associative. That is, the expression `x :: xs` is actually the method call `xs.::(x)`, which in turn calls the data constructor `::(x,xs)`. See the Scala language spec ification for more information.

- `scanLeft` and `scanRight`—Like `foldLeft` and `foldRight`, but they return the `List` of partial results rather than just the final accumulated value

We recommend that you look through the Scala API documentation after finishing this chapter, to see what other functions there are. If you find yourself writing an explicit recursive function for doing some sort of list manipulation, check the `List` API to see if something like the function you need already exists.

3.4.2 Loss of efficiency when assembling list functions from simpler components

One of the problems with `List` is that, although we can often express operations and algorithms in terms of very general-purpose functions, the resulting implementation isn't always efficient—we may end up making multiple passes over the same input, or else have to write explicit recursive loops to allow early termination.

EXERCISE 3.24

Hard: As an example, implement `hasSubsequence` for checking whether a `List` con tains another `List` as a subsequence. For instance, `List(1,2,3,4)` would have `List(1,2)`, `List(2,3)`, and `List(4)` as subsequences, among others. You may have some difficulty finding a concise purely functional implementation that is also effi cient. That's okay. Implement the function however comes most naturally. We'll return to this implementation in chapter 5 and hopefully improve on it. Note: Any two values `x` and `y` can be compared for equality in Scala using the expression `x == y`.

```
def hasSubsequence[A](sup: List[A], sub: List[A]): Boolean
```

3.5 Trees

`List` is just one example of what's called an *algebraic data type (ADT)*. (Somewhat con fusingly, ADT is sometimes used elsewhere to stand for *abstract data type*.) An ADT is just a data type defined by one or more data constructors, each of which may contain zero or more arguments. We say that the data type is the *sum* or *union* of its data construc tors, and each data constructor is the *product* of its arguments, hence the name *alge braic* data type.[14]

[14] The naming is not coincidental. There's a deep connection, beyond the scope of this book, between the "addition" and "multiplication" of types to form an ADT and addition and multiplication of numbers.

Tuple types in Scala

Pairs and tuples of other arities are also algebraic data types. They work just like the ADTs we've been writing here, but have special syntax:

```
scala> val p = ("Bob", 42)
p: (java.lang.String, Int) = (Bob,42)

scala> p._1
res0: java.lang.String = Bob

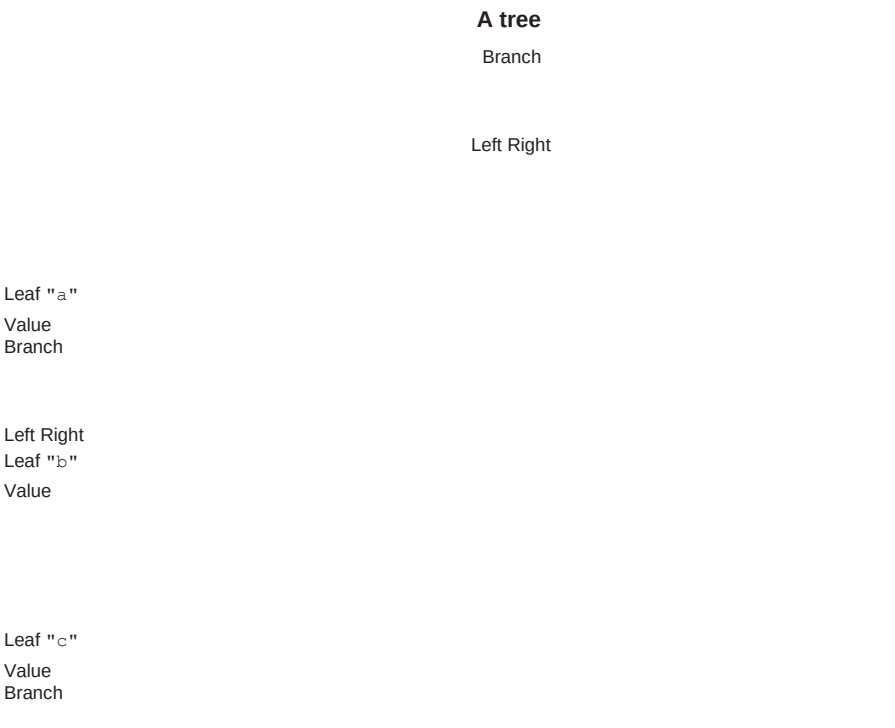
scala> p._2
res1: Int = 42

scala> p match { case (a,b) => b }
res2: Int = 42
```

In this example, `("Bob", 42)` is a pair whose type is `(String,Int)`, which is syn tactic sugar for `Tuple2[String,Int]` (API link: http://mng.bz/1F2N). We can extract the first or second element of this pair (a `Tuple3` will have a method `_3`, and so on), and we can pattern match on this pair much like any other `case class`. Higher arity tuples work similarly—try experimenting with them in the REPL if you're interested.

Algebraic data types can be used to define other data structures. Let's define a simple binary tree data structure:

```
sealed trait Tree[+A]
case class Leaf[A](value: A) extends Tree[A]
case class Branch[A](left: Tree[A], right: Tree[A]) extends Tree[A]
```



Left Right

Leaf "d"
Value

Pattern matching again provides a convenient way of operating over elements of our ADT. Let's try writing a few functions.

EXERCISE 3.25 Write a function `size` that counts the number of nodes (leaves and branches) in a tree.

EXERCISE 3.26

Write a function `maximum` that returns the maximum element in a `Tree[Int]`. (Note: In Scala, you can use `x.max(y)` or `x max y` to compute the maximum of two integers `x` and `y`.)

EXERCISE 3.27

Write a function `depth` that returns the maximum path length from the root of a tree to any leaf.

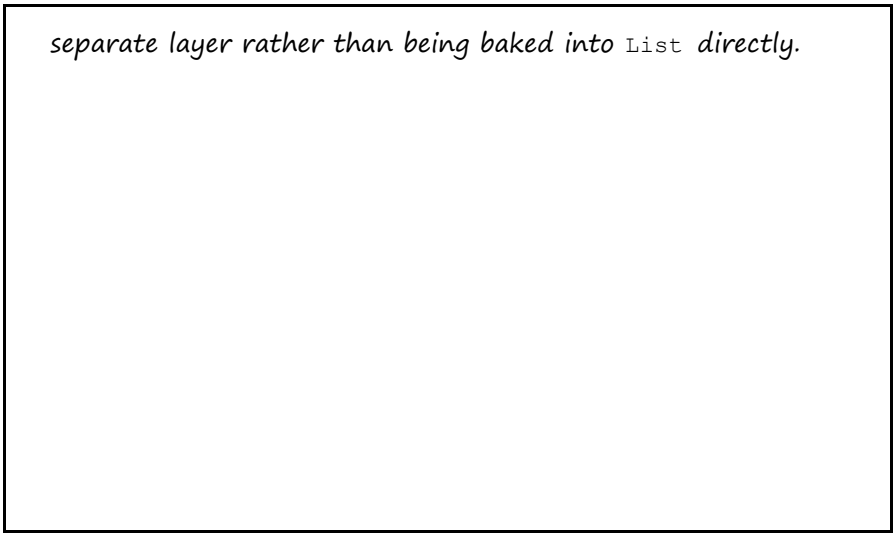
EXERCISE 3.28

Write a function `map`, analogous to the method of the same name on `List`, that modifies each element in a tree with a given function.

ADTs and encapsulation

One might object that algebraic data types violate encapsulation by making public the internal representation of a type. In FP, we approach concerns about encapsulation differently—we don't typically have delicate mutable state which could lead to bugs or violation of invariants if exposed publicly. Exposing the data constructors of a type is often fine, and the decision to do so is approached much like any other decision about what the public API of a data type should be.[15]

We do typically use ADTs for situations where the set of cases is closed (known to be fixed). For `List` *and* `Tree`*, changing the set of data constructors would significantly change what these data types are.* `List` *is a singly linked list—that is its nature—and the two cases* `Nil` *and* `Cons` *form part of its useful public API. We can certainly write code that deals with a more abstract API than* `List` *(we'll see examples of this later in the book), but this sort of information hiding can be handled as a*



[15] It's also possible in Scala to expose patterns like `Nil` and `Cons` independent of the actual data constructors of the type.

EXERCISE 3.29

Generalize `size`, `maximum`, `depth`, and `map`, writing a new function `fold` that abstracts over their similarities. Reimplement them in terms of this more general function. Can you draw an analogy between this `fold` function and the left and right folds for `List`?

3.6 Summary

In this chapter, we covered a number of important concepts. We introduced algebraic data types and pattern matching, and showed how to implement purely functional data structures, including the singly linked list. We hope that, through the exercises in this chapter, you got more comfortable writing pure functions and generalizing them. We'll continue to develop these skills in the chapters ahead.

Handling errors without exceptions

We noted briefly in chapter 1 that throwing exceptions is a side effect. If exceptions aren't used in functional code, what *is* used instead? In this chapter, we'll learn the basic principles for raising and handling errors functionally. The big idea is that we can represent failures and exceptions with ordinary values, and we can write higher-order functions that abstract out common patterns of error handling and recovery. The functional solution, of

returning errors as values, is safer and retains referential transparency, and through the use of higher-order functions, we can preserve the primary benefit of exceptions—*consolidation of error-handling logic.* We'll see how this works over the course of this chapter, after we take a closer look at exceptions and discuss some of their problems.

For the same reason that we created our own `List` data type in the previous chapter, we'll re-create in this chapter two Scala standard library types: `Option` and `Either`. The purpose is to enhance your understanding of how these types can be used for handling errors. After completing this chapter, you should feel free to use the Scala standard library version of `Option` and `Either` (though you'll notice that the standard library versions of both types are missing some of the useful functions we define in this chapter).

4.1 The good and bad aspects of exceptions

Why do exceptions break referential transparency, and why is that a problem? Let's look at a simple example. We'll define a function that throws an exception and call it.

`val y: Int = ...` declares `y`

```
def failingFn(i: Int): Int = {
val y: Int = throw new Exception("fail!") try {
val x = 42 + 5
x+y
}
catch { case e: Exception => 43 } }
```

as having type `Int` and sets it equal to the right-hand side of `=`.

A `catch` block is just a pattern matching block like the ones we've seen. `case e: Exception` is a pattern that matches any `Exception`, and it binds this value to the identifier `e`. The match returns the value `43`.

Calling `failingFn` from the REPL gives the expected error:

```
scala> failingFn(12)
java.lang.Exception: fail!
  at .failingFn(<console>:8)
   ...
```

We can prove that `y` is not referentially transparent. Recall that any RT expression may be substituted with the value it refers to, and this substitution should preserve pro gram meaning. If we substitute `throw new Exception("fail!")` for `y` in `x + y`, it pro duces a different result, because the exception will now be raised inside a `try` block that will catch the exception and return `43`:

```
def failingFn2(i: Int): Int = {
try {
val x = 42 + 5
x + ((throw new Exception("fail!")): Int) }
catch { case e: Exception => 43 }
}
```

We can demonstrate this in the REPL:

```
scala> failingFn2(12)
res1: Int = 43
```

A thrown `Exception` can be given any type; here we're annotating it with the type `Int`.

Another way of understanding RT is that the meaning of RT expressions *does not depend on context* and may be reasoned about locally, whereas the meaning of non-RT expres sions is *context-dependent* and requires more global reasoning. For instance, the mean ing of the RT expression `42 + 5` doesn't depend on the larger expression it's embedded in—it's always and forever equal to `47`. But the meaning of the expression `throw new Exception("fail")` is very context-dependent—as we just demonstrated, it takes on different meanings depending on which `try` block (if any) it's nested within.

There are two main problems with exceptions:

- As we just discussed, *exceptions break RT and introduce context dependence*, moving us away from the simple reasoning of the substitution model and making it possible to write confusing exception-based code. This is the source of the folklore advice that exceptions should be used only for error handling, not for control flow.

- *Exceptions are not type-safe.* The type of `failingFn`, `Int => Int` tells us nothing about the fact that exceptions may occur, and the compiler will certainly not force callers of `failingFn` to make a decision about how to handle those excep tions. If we forget to check for an exception in `failingFn`, this won't be detected until runtime.

Checked exceptions

Java's checked exceptions at least force a decision about whether to handle or reraise an error, but they result in significant boilerplate for callers. More importantly, they don't work for higher-order functions, which can't possibly be aware of the spe cific exceptions that could be raised by their arguments. For example, consider the `map` function we defined for `List`:

```
def map[A,B](l: List[A])(f: A => B): List[B]
```

This function is clearly useful, highly generic, and at odds with the use of checked exceptions—we can't have a version of `map` for every single checked exception that could possibly be thrown by `f`. Even if we wanted to do this, how would `map` even know what exceptions were possible? This is why generic

code, even in Java, so often resorts to using `RuntimeException` or some common checked `Exception` type.

We'd like an alternative to exceptions without these drawbacks, but we don't want to lose out on the primary benefit of exceptions: they allow us to *consolidate and centralize error-handling logic*, rather than being forced to distribute this logic throughout our codebase. The technique we use is based on an old idea: instead of throwing an excep tion, we return a value indicating that an exceptional condition has occurred. This idea might be familiar to anyone who has used return codes in C to handle excep tions. But instead of using error codes, we introduce a new generic type for these "pos sibly defined values" and use higher-order functions to encapsulate common patterns of handling and propagating errors. Unlike C-style error codes, the error-handling strategy we use is *completely type-safe*, and we get full assistance from the type-checker in forcing us to deal with errors, with a minimum of syntactic noise. We'll see how all of this works shortly.

4.2 Possible alternatives to exceptions

Let's now consider a realistic situation where we might use an exception and look at different approaches we could use instead. Here's an implementation of a function that computes the mean of a list, which is undefined if the list is empty:

`Seq` is the common interface of various linear sequence-like collections. Check the API docs

```
def mean(xs: Seq[Double]): Double = if (xs.isEmpty)
```

`sum` is defined as a method on `Seq` only if the elements of the sequence are numeric. The standard library accomplishes this trick with implicits, which we won't go into here.

(http://mng.bz/f4k9) for more information.

```
throw new ArithmeticException("mean of empty list!") else xs.sum / xs.length
```


The `mean` function is an example of what's called a *partial function*: it's not defined for some inputs. A function is typically partial because it makes some assumptions about its inputs that aren't implied by the input types.[1] You may be used to throwing exceptions in this case, but we have a few other options. Let's look at these for our `mean` example.

The first possibility is to return some sort of bogus value of type `Double`. We could simply return `xs.sum / xs.length` in all cases, and have it result in `0.0/0.0` when the input is empty, which is `Double.NaN`; or we could return some other sentinel value. In other situations, we might return `null` instead of a value of the needed type. This gen eral class of approaches is how error handling is often

done in languages without exceptions, and we reject this solution for a few reasons:

- It allows errors to silently propagate—the caller can forget to check this condi tion and won't be alerted by the compiler, which might result in subsequent code not working properly. Often the error won't be detected until much later in the code.
- Besides being error-prone, it results in a fair amount of boilerplate code at call sites, with explicit `if` statements to check whether the caller has received a "real" result. This boilerplate is magnified if you happen to be calling several functions, each of which uses error codes that must be checked and aggregated in some way.
- It's not applicable to polymorphic code. For some output types, we might not even *have* a sentinel value of that type even if we wanted to! Consider a function like `max`, which finds the maximum value in a sequence according to a custom comparison function: `def max[A](xs: Seq[A])(greater: (A,A) => Boolean): A`. If the input is empty, we can't invent a value of type `A`. Nor can `null` be used here, since `null` is only valid for non-primitive types, and `A` may in fact be a primitive like `Double` or `Int`.
- It demands a special policy or calling convention of callers—proper use of the `mean` function would require that callers do something other than call `mean` and make use of the result. Giving functions special policies like this makes it difficult to pass them to higher-order functions, which must treat all arguments uniformly.

The second possibility is to force the caller to supply an argument that tells us what to do in case we don't know how to handle the input:

```
def mean_1(xs: IndexedSeq[Double], onEmpty: Double): Double =
          if (xs.isEmpty) onEmpty
          else xs.sum / xs.length
```

This makes `mean` into a total function, but it has drawbacks—it requires that *immediate* callers have direct knowledge of how to handle the undefined case and limits them to returning a `Double`. What if `mean` is called as part of a larger computation and we'd like to abort that computation if `mean` is undefined? Or perhaps we'd like to take some completely different branch in the larger computation in this case? Simply passing an `onEmpty` parameter doesn't give us this freedom.

We need a way to defer the decision of how to handle undefined cases so that they can be dealt with at the most appropriate level.

[1] A function may also be partial if it doesn't terminate for some inputs. We won't discuss this form of partiality here, since it's not a recoverable error so there's no question of how best to handle it. See the chapter notes for more about partiality.

4.3 The Option data type

The solution is to represent explicitly in the return type that a function may not always have an answer. We can think of this as deferring to the caller for the error-handling strategy. We introduce a new type, `Option`. As we mentioned earlier, this type also exists in the Scala standard library, but we're re-creating it here for pedagogical purposes:

```
sealed trait Option[+A]
case class Some[+A](get: A) extends Option[A]
case object None extends Option[Nothing]
```

`Option` has two cases: it can be defined, in which case it will be a `Some`, or it can be undefined, in which case it will be `None`.

We can use `Option` for our definition of `mean` like so:

```
def mean(xs: Seq[Double]): Option[Double] =
  if (xs.isEmpty) None
  else Some(xs.sum / xs.length)
```

The return type now reflects the possibility that the result may not always be defined. We still always return a result of the declared type (now `Option[Double]`) from our function, so `mean` is now a *total function*. It takes each value of the input type to exactly one value of the output type.

Responding to invalid inputs

Using a sentinel value

.
.
.

.
.
.

Using the Option type

.
.
.
5
4
3
2
1

0
–1 –2
Valid inputs

Invalid inputs

`logID`
Valid
outputs

Sentinel value
.
.699
.602
.478
.301
0

–9999999.99
.
5

4
3
2
1

0
–1 –2
Valid inputs

Invalid inputs

`logID`
Valid
outputs

Invalid output
.
Some(.699) Some(.602) Some(.478) Some(.301) Some(0)

None

`logID: Double => Double`

Mapping all invalid inputs to a special value of the same type as the valid outputs. Ambiguous, and compiler can't check that caller handles it correctly.

`logID: Double => Option[Double]`

Every valid output is wrapped in Some. Invalid inputs are mapped to None. The compiler forces the caller to deal explicitly with the possibility of failure.

4.3.1 Usage patterns for Option

Partial functions abound in programming, and `Option` (and the `Either` data type that we'll discuss shortly) is typically how this partiality is dealt with in FP. You'll see `Option` used throughout the Scala standard library, for instance:

- Map lookup for a given key (http://mng.bz/ha64) returns `Option`.
- `headOption` and `lastOption` defined for lists and other iterables (http:// mng.bz/Pz86) return an `Option` containing the first or last elements of a sequence if it's nonempty.

These aren't the only examples—we'll see `Option` come up in many different situa tions. What makes `Option` convenient is that we can factor out common patterns of error handling via higher-order functions, freeing us from writing the usual boiler plate that comes with exception-handling code. In this section, we'll cover some of the basic functions for working with `Option`. Our goal is not for you to attain fluency with all these functions, but just to get you familiar enough that you can revisit this chapter and make progress on your

own when you have to write some functional code to deal with errors.

BASIC FUNCTIONS ON OPTION

`Option` can be thought of like a `List` that can contain at most one element, and many of the `List` functions we saw earlier have analogous functions on `Option`. Let's look at some of these functions.

We'll do something slightly different than in chapter 3 where we put all our `List` functions in the `List` companion object. Here we'll place our functions, when possi ble, inside the body of the `Option` trait, so they can be called with the syntax `obj.fn(arg1)` or `obj fn arg1` instead of `fn(obj, arg1)`. This is a stylistic choice with no real significance, and we'll use both styles throughout this book.[2]

This choice raises one additional complication with regard to variance that we'll discuss in a moment. Let's take a look.

Apply `f` if the `Option` is not `None`.

Don't evaluate `ob` unless needed.

```
trait Option[+A] {
def map[B](f: A => B): Option[B]
def flatMap[B](f: A => Option[B]): Option[B] def getOrElse[B >: A](default: => B): B
def orElse[B >: A](ob: => Option[B]): Option[B] def filter(f: A => Boolean): Option[A]
}
```

Convert `Some` to `None` if the value doesn't satisfy `f`.

Apply `f`, which may fail, to the `Option` if not `None`.

The `B >: A` says that the `B` type parameter must be a supertype of `A`.

[2] In general, we'll use this object-oriented style of syntax where possible for functions that have a single, clear operand (like `List.map`), and the standalone function style otherwise.

There is some new syntax here. The `default: => B` type annotation in `getOrElse` (and the similar annotation in `orElse`) indicates that the argument is of type `B`, but won't be evaluated until it's needed by the function. Don't worry about this for now—we'll talk much more about this concept of *non-strictness* in the next chapter. Also, the `B >: A` type parameter on the `getOrElse` and `orElse` functions indicates that `B` must be equal to or a *supertype* of `A`. It's needed to convince Scala that it's still safe to declare `Option[+A]` as covariant in `A`. See the chapter notes for more detail—it's unfortunately somewhat complicated, but a necessary complication in Scala. Fortunately, fully

understanding subtyping and variance isn't essential for our purposes here.

EXERCISE 4.1

Implement all of the preceding functions on `Option`. As you implement each function, try to think about what it means and in what situations you'd use it. We'll explore when to use each of these functions next. Here are a few hints for solving this exercise:

- It's fine to use pattern matching, though you should be able to implement all the functions besides `map` and `getOrElse` without resorting to pattern matching.
- For `map` and `flatMap`, the type signature should be enough to determine the implementation.
- `getOrElse` returns the result inside the `Some` case of the `Option`, or if the `Option` is `None`, returns the given default value.
- `orElse` returns the first `Option` if it's defined; otherwise, it returns the second `Option`.

USAGE SCENARIOS FOR THE BASIC OPTION FUNCTIONS

Although we can explicitly pattern match on an `Option`, we'll almost always use the above higher-order functions. Here, we'll try to give some guidance for when to use each one. Fluency with these functions will come with practice, but the objective here is to get some basic familiarity. Next time you try writing some functional code that uses `Option`, see if you can recognize the patterns these functions encapsulate before you resort to pattern matching.

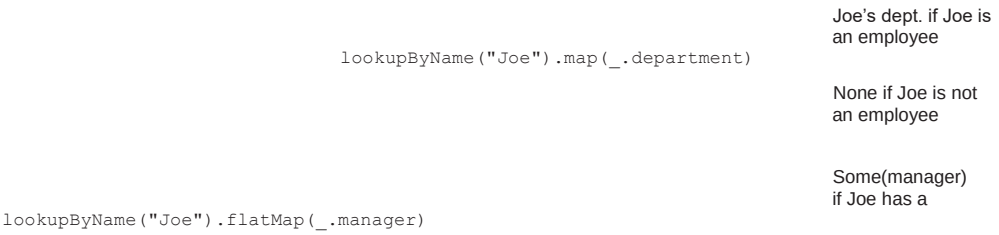
Let's start with `map`. The `map` function can be used to transform the result inside an `Option`, if it exists. We can think of it as proceeding with a computation on the assumption that an error hasn't occurred; it's also a way of deferring the error han dling to later code:

```
case class Employee(name: String, department: String)

def lookupByName(name: String): Option[Employee] = ...

val joeDepartment: Option[String] =
  lookupByName("Joe").map(_.department)
```


Here, `lookupByName("Joe")` returns an `Option[Employee]`, which we transform using `map` to pull out the `Option[String]` representing the department. Note that we don't need to explicitly check the result of `lookupByName("Joe")`; we simply continue the computation as if no error occurred, inside the argument to `map`. If `employeesBy Name.get("Joe")` returns `None`, this will abort the rest of the computation and `map` will not call the `_.department` function at all.



```
lookupByName("Joe").map(_.department).getOrElse("Default Dept.")
```
manager

None if Joe is not an employee or doesn't have a manager

Joe's department if he has one

"Default Dept." if not

`flatMap` is similar, except that the function we provide to transform the result can itself fail.

EXERCISE 4.2

Implement the `variance` function in terms of `flatMap`. If the mean of a sequence is `m`, the variance is the mean of `math.pow(x - m, 2)` for each element `x` in the sequence. See the definition of variance on Wikipedia (http://mng.bz/0Qsr).

```
def variance(xs: Seq[Double]): Option[Double]
```

As the implementation of `variance` demonstrates, with `flatMap` we can construct a computation with multiple stages, any of which may fail, and the computation will abort as soon as the first failure is encountered, since `None.flatMap(f)` will immedi ately return `None`, without running `f`.

We can use `filter` to convert successes into failures if the successful values don't match the given predicate. A common pattern is to transform an `Option` via calls to `map`, `flatMap`, and/or `filter`, and then use `getOrElse` to do error handling at the end:

```
val dept: String =
  lookupByName("Joe").
  map(_.dept).
  filter(_ != "Accounting").
  getOrElse("Default Dept")
```

`getOrElse` is used here to convert from an `Option[String]` to a `String`, by providing a default department in case the key `"Joe"` didn't exist in the `Map` or if Joe's depart ment was `"Accounting"`.

`orElse` is similar to `getOrElse`, except that we return another `Option` if the first is undefined. This is often useful when we need to chain together possibly failing com putations, trying the second if the first hasn't succeeded.

A common idiom is to do `o.getOrElse(throw new Exception("FAIL"))` to con vert the `None` case of an `Option` back to an exception. The general rule of thumb is that we use exceptions only if no reasonable program would ever catch the exception; if for some callers the exception might be a recoverable error, we use `Option` (or `Either`, discussed later) to give them flexibility.

As you can see, returning errors as ordinary values can be convenient and the use of higher-order functions lets us achieve the same sort of consolidation of error handling logic we would get from using exceptions. Note that we don't have to check for `None` at each stage of the computation—we can apply several transformations and then check for and handle `None` when we're ready. But we also get additional safety, since `Option[A]` is a different type than `A`, and the compiler won't let us forget to explicitly defer or handle the possibility of `None`.

4.3.2 Option composition, lifting, and wrapping exception-oriented APIs

It may be easy to jump to the conclusion that once we start using `Option`, it infects our entire code base. One can imagine how any callers of methods that take or return `Option` will have to be modified to handle either `Some` or `None`. But this doesn't hap pen, and the reason is that we can *lift* ordinary functions to become functions that operate on `Option`.

For example, the `map` function lets us operate on values of type `Option[A]` using a function of type `A => B`, returning `Option[B]`. Another way of looking at this is that `map` turns a function `f` of type `A => B` into a function of type `Option[A] => Option[B]`. Let's make this explicit:

```
def lift[A,B](f: A => B): Option[A] => Option[B] = _ map f
```

This tells us that any function that we already have lying around can be transformed (via `lift`) to operate *within the context of* a single `Option` value. Let's look at an example:

```
val absO: Option[Double] => Option[Double] = lift(math.abs)
```


The `math` object contains various standalone mathematical functions including `abs`, `sqrt`, `exp`, and so on. We didn't need to rewrite the `math.abs` function to work with optional values; we just lifted it into the `Option` context after the fact. We can do this for *any* function. Let's look at another example. Suppose we're implementing the logic for a car insurance company's website, which contains a page where users can submit a form to request an instant online quote. We'd like to parse the information from this form and ultimately call our rate function:

```
/**
 * Top secret formula for computing an annual car
 * insurance premium from two key factors.
 */
def insuranceRateQuote(age: Int, numberOfSpeedingTickets: Int): Double
```

We want to be able to call this function, but if the user is submitting their age and number of speeding tickets in a web form, these fields will arrive as simple strings that we have to (try to) parse into integers. This parsing may fail; given a string, `s`, we can attempt to parse it into an `Int` using `s.toInt`, which throws a `NumberFormat Exception` if the string isn't a valid integer:

```
scala> "112".toInt
res0: Int = 112

scala> "hello".toInt
java.lang.NumberFormatException: For input string: "hello"
  at java.lang.NumberFormatException.forInputString(...)
  ...
```

Lifting functions

```
lift(math.abs): Option[Double] => Option[Double]
```

```
math.abs: Double => Double
```

`lift(f)` returns a function which maps `None` to `None` and applies `f` to the contents of `Some`. `f` need not be aware of the `Option` type at all.

Let's convert the exception-based API of `toInt` to `Option` and see if we can implement a function `parseInsuranceRateQuote`, which

takes the age and number of speeding tickets as strings, and attempts calling the `insuranceRateQuote` function if parsing both values is successful.

```
def parseInsuranceRateQuote(
age: String,
numberOfSpeedingTickets: String): Option[Double] = { val optAge: Option[Int] =
Try(age.toInt)
```

The `toInt` method is available on any `String`.

```
    val optTickets: Option[Int] = Try(numberOfSpeedingTickets.toInt)
```


Note: This discards information about the error. We'll improve on this later in the chapter.

```
insuranceRateQuote(optAge, optTickets) }
```

```
def Try[A](a: => A): Option[A] = try Some(a)
catch { case e: Exception => None }
```

Doesn't type check! See following discussion.

We accept the `A` argument non-strictly, so we can catch any exceptions that occur while evaluating `a` and convert them to `None`.

The `Try` function is a general-purpose function we can use to convert from an excep tion-based API to an `Option`-oriented API. This uses a non-strict or lazy argument, as indicated by the `=> A` as the type of `a`. We'll discuss laziness much more in the next chapter.

But there's a problem—after we parse `optAge` and `optTickets` into `Option[Int]`, how do we call `insuranceRateQuote`, which currently takes two `Int` values? Do we have to rewrite `insuranceRateQuote` to take `Option[Int]` values instead? No, and changing `insuranceRateQuote` would be entangling concerns, forcing it to be aware that a prior computation may have failed, not to mention that we may not have the ability to modify `insuranceRateQuote`—perhaps it's defined in a separate module that we don't have access to. Instead, we'd like to lift `insuranceRateQuote` to operate in the context of two optional values. We could do this using explicit pattern matching in the body of `parseInsuranceRateQuote`, but that's going to be tedious.

EXERCISE 4.3

Write a generic function `map2` that combines two `Option` values using a binary func tion. If either `Option` value is `None`, then the return value is too. Here is its signature:

```
def map2[A,B,C](a: Option[A], b: Option[B])(f: (A, B) => C): Option[C]
```

With `map2`, we can now implement `parseInsuranceRateQuote`:

Functions accepting a single argument may

```
def parseInsuranceRateQuote( age: String,
```

be called with braces instead of parentheses; this is equivalent to `Try(age.toInt)`.

If either parse fails, this will immediately return `None`.

```
numberOfSpeedingTickets: String): Option[Double] = {
val optAge: Option[Int] = Try { age.toInt }
val optTickets: Option[Int] = Try { numberOfSpeedingTickets.toInt } map2(optAge,
optTickes)(insuranceRateQuote)
}
```

The `map2` function means that we never need to modify any existing functions of two arguments to make them "`Option`-aware." We can lift them to operate in the context of `Option` after the fact. Can you already see how you might define `map3`, `map4`, and `map5`? Let's look at a few other similar cases.

EXERCISE 4.4

Write a function `sequence` that combines a list of `Option`s into one `Option` containing a list of all the `Some` values in the original list. If the original list contains `None` even once, the result of the function should be `None`; otherwise the result should be `Some` with a list of all the values. Here is its signature:[3]

```
def sequence[A](a: List[Option[A]]): Option[List[A]]
```

Sometimes we'll want to map over a list using a function that might fail, returning `None` if applying it to any element of the list returns `None`. For example, what if we have a whole list of `String` values that we wish to parse to `Option[Int]`? In that case, we can simply sequence the results of the `map`:

```
def parseInts(a: List[String]): Option[List[Int]] =
  sequence(a map (i => Try(i.toInt)))
```

Unfortunately, this is inefficient, since it traverses the list twice, first to convert each `String` to an `Option[Int]`, and a second pass to combine these `Option[Int]` values into an `Option[List[Int]]`. Wanting to sequence the results of a `map` this way is a common enough occurrence to warrant a new generic function, `traverse`, with the following signature:

```
def traverse[A, B](a: List[A])(f: A => Option[B]): Option[List[B]]
```

EXERCISE 4.5

Implement this function. It's straightforward to do using `map` and `sequence`, but try for a more efficient implementation that only looks at the list once. In fact, imple ment `sequence` in terms of `traverse`.

For-comprehensions

Since lifting functions is so common in Scala, Scala provides a syntactic construct called the for-comprehension that it expands automatically to a series of `flatMap` and `map` calls. Let's look at how `map2` could be implemented with for-comprehensions.

[3] This is a clear instance where it's not appropriate to define the function in the OO style. This shouldn't be a method on `List` (which shouldn't need to know anything about `Option`), and it can't be a method on `Option`, so it goes in the `Option` companion object.